מערכות הפעלה

ניהול זיכרון

ניהול זיכרון

3.1 Basic memory management (חלוקה למחיצות קבועות)

3.2 Swapping (חלוקה למחיצות נעות)

3.3 Virtual memory (paging)

3.4 Page replacement algorithms

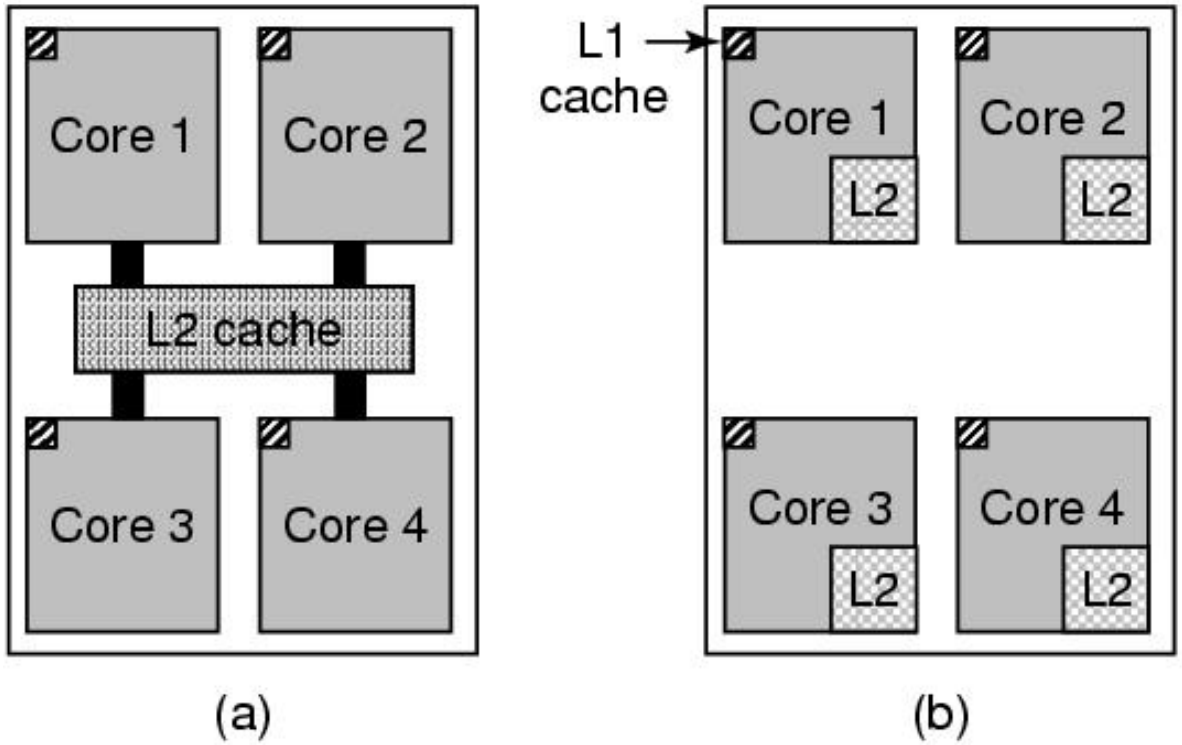
3.5 Design issues for paging systems

3.6 Implementation issues

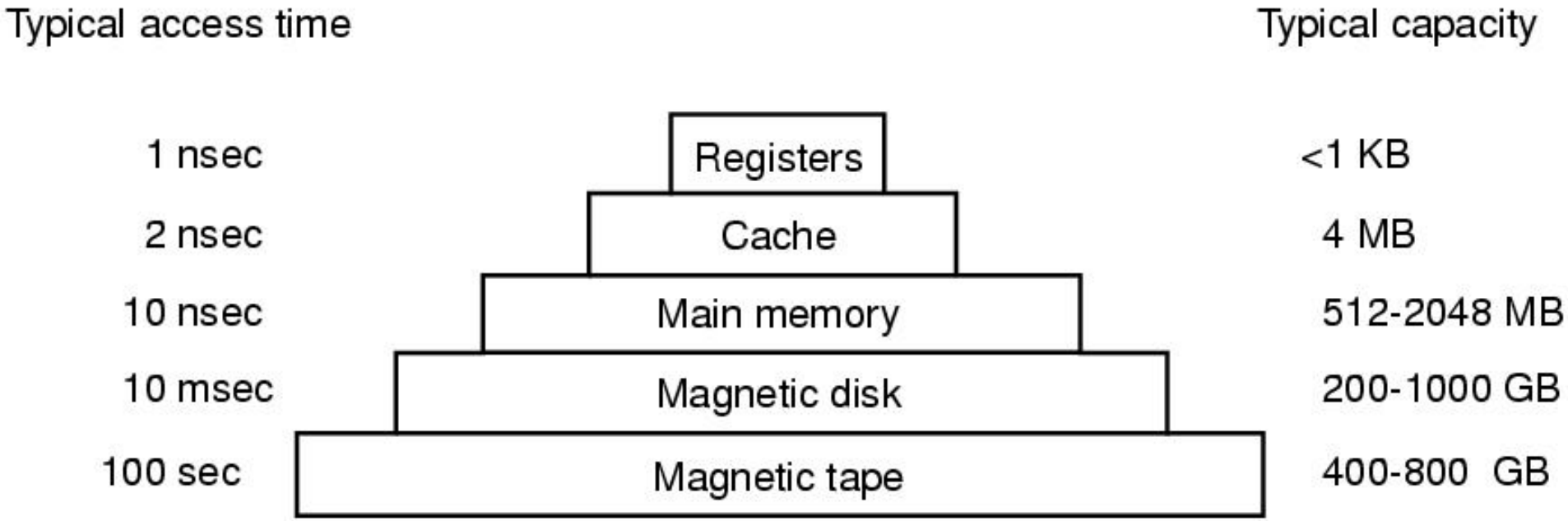
3.7 Segmentation

ניהול זיכרון

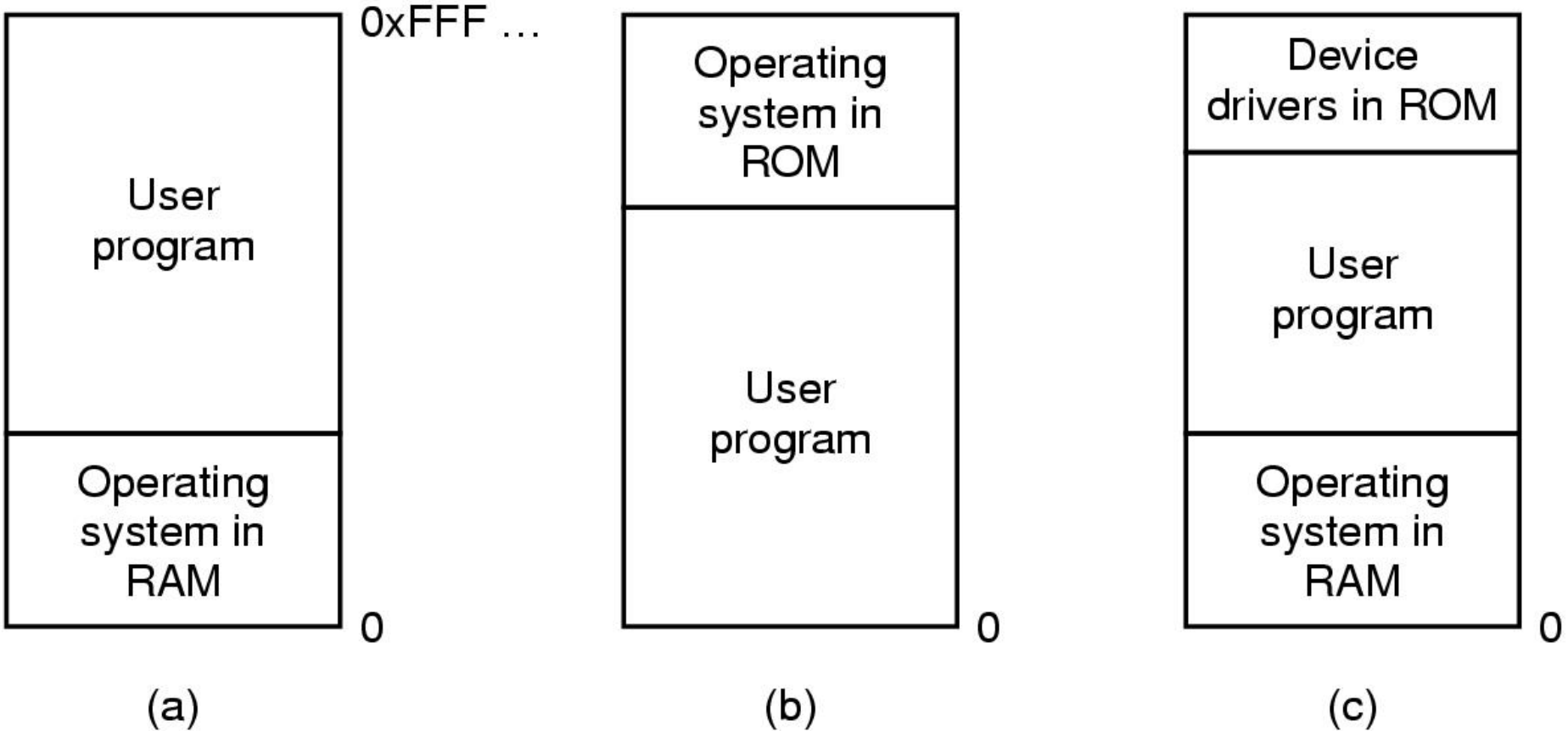
Multithreaded and Multicore Chips



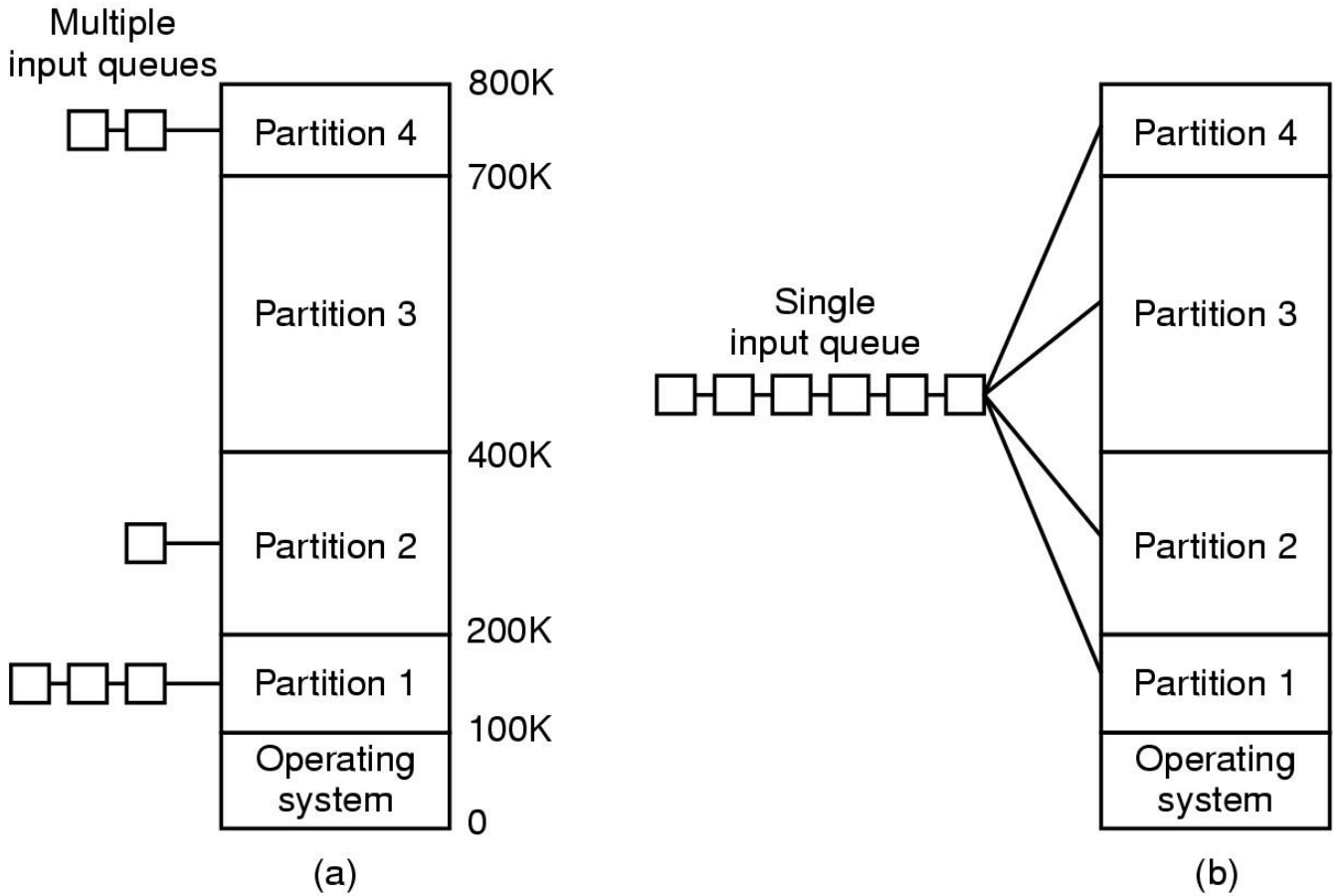
Computer Hardware Review



ניהול זיכרון בסיסי



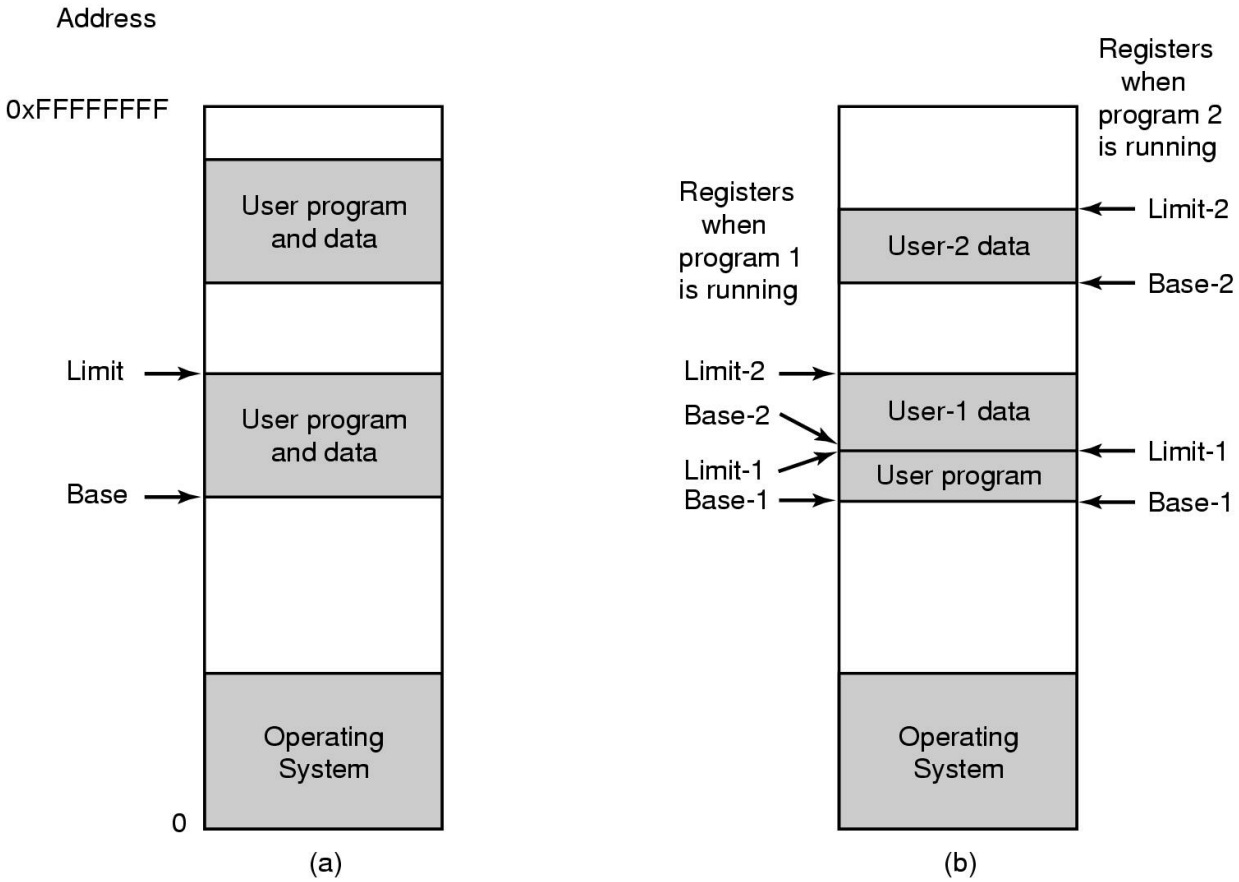
Multiprogramming with Fixed Partitions



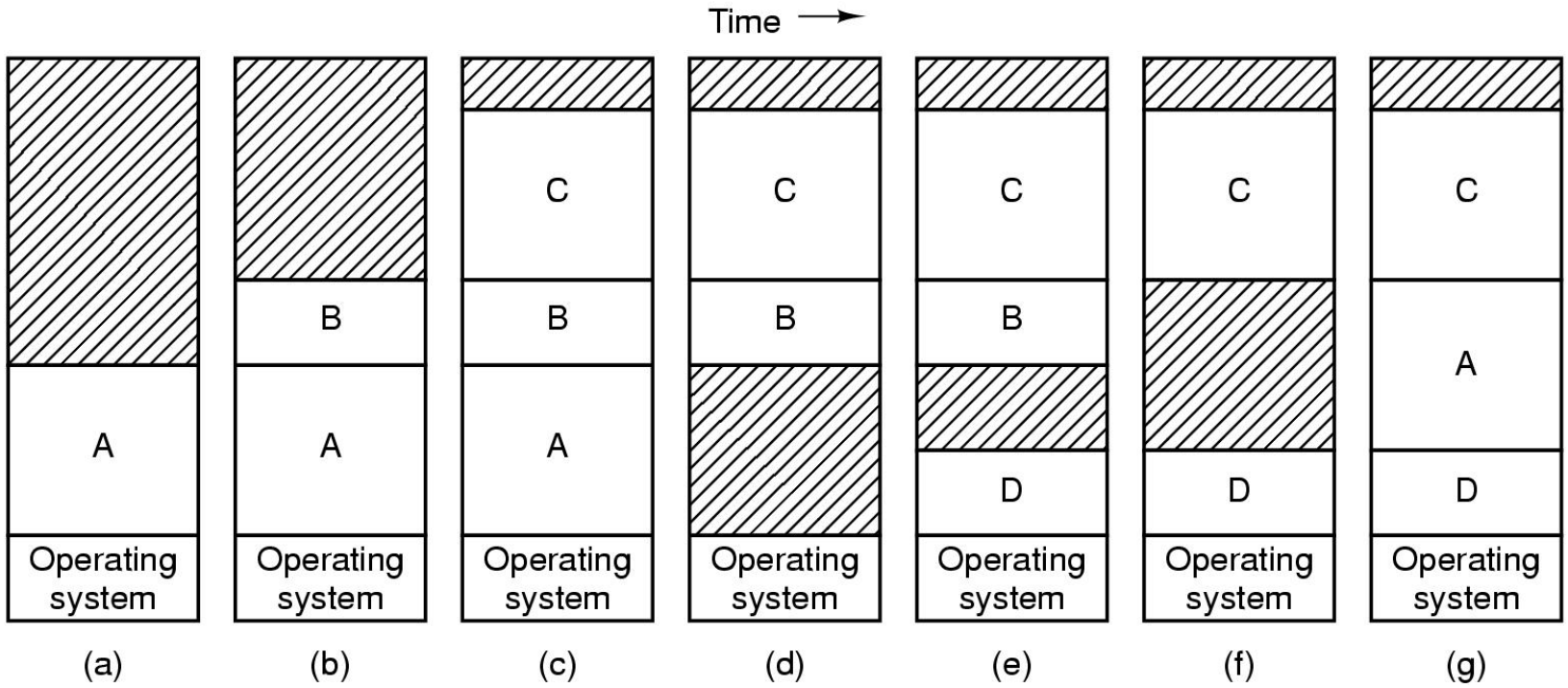
Relocation and Protection

- Cannot be sure where program will be loaded in memory
  - address locations of variables, code routines cannot be absolute
  - must keep a program out of other processes' partitions
- Use base and limit values
  - address locations added to base value to map to physical addr
  - address locations larger than limit value is an error

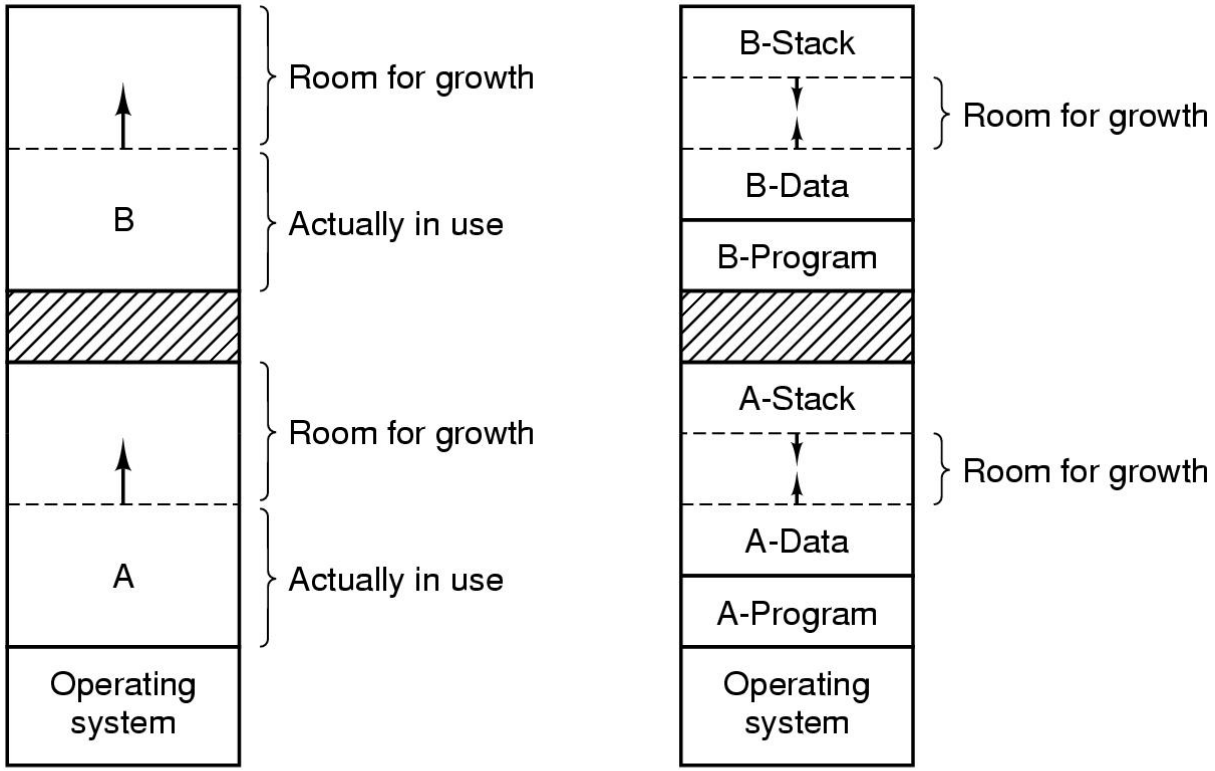
Computer


Address
0xFFFFFFFF
User program and data
Limit
User program and data
Base
Operating System
0
(a)
Registers when program 2 is running
Registers when program 1 is running
Limit-2
User-2 data
Base-2
Limit-2
Base-2
User-1 data
Limit-1
Limit-1
User program
Base-1
Base-1
Operating System
(b)

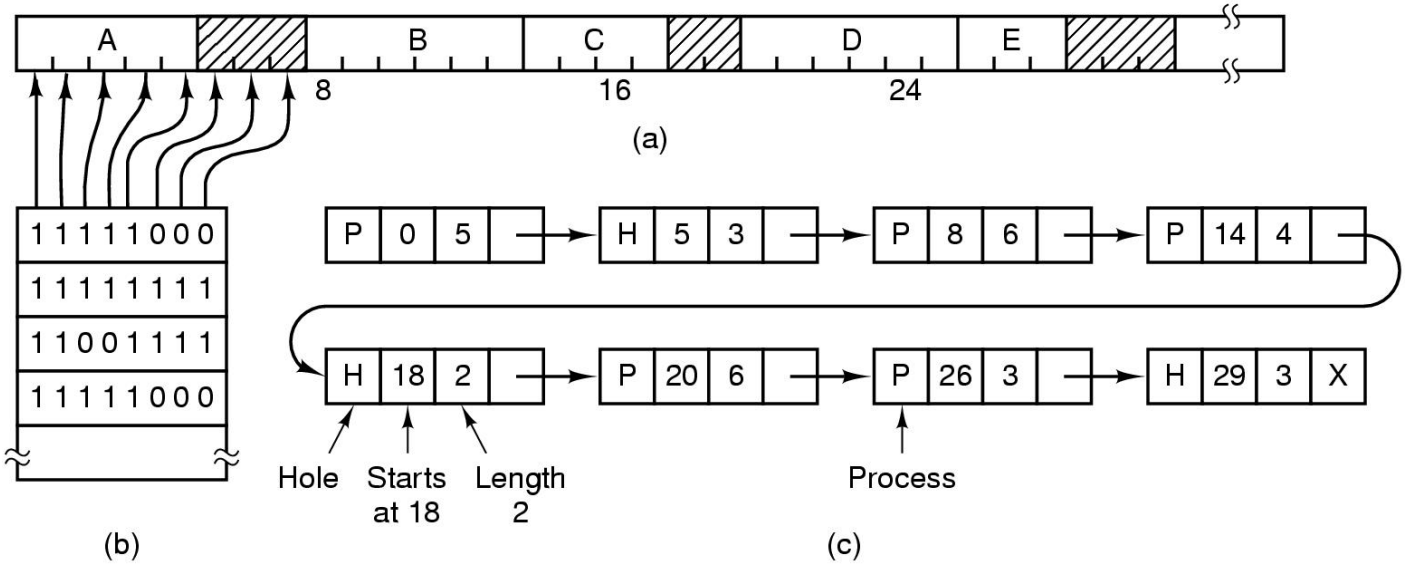
Swapping



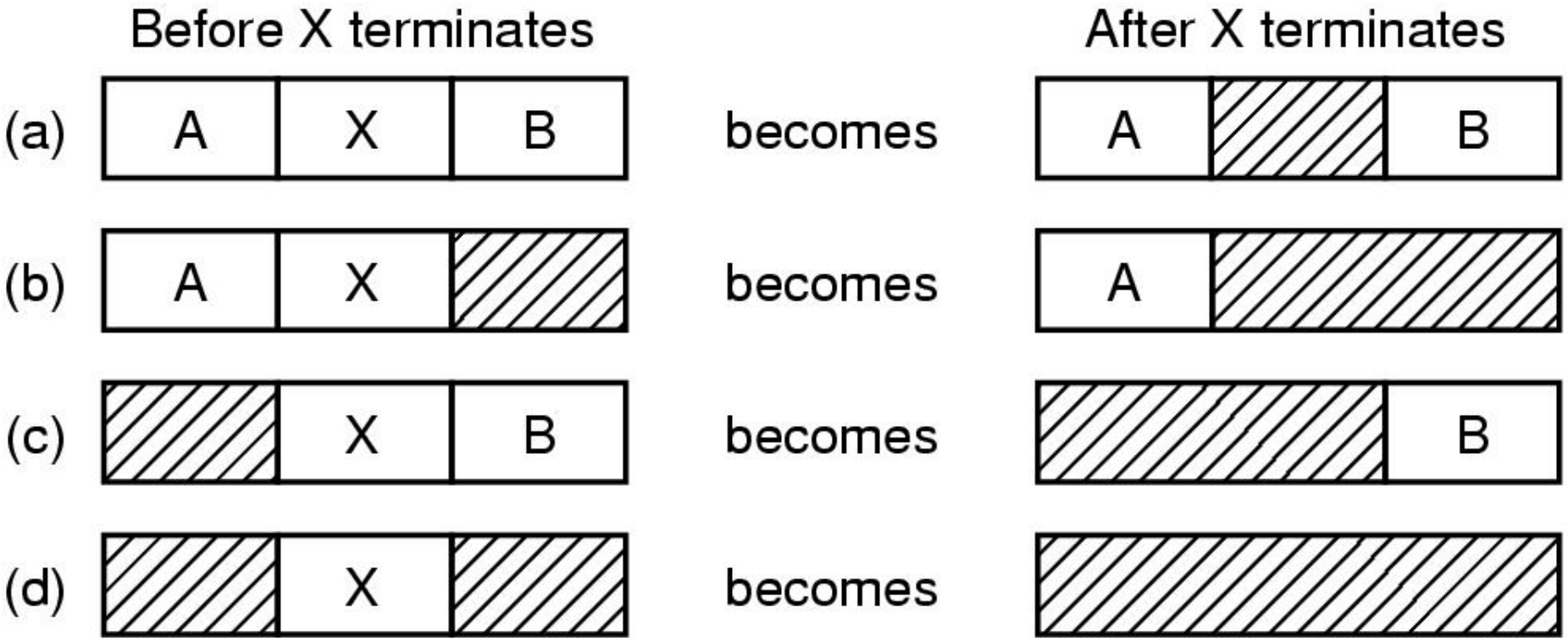
Swapping


Room for growth
B
Actually in use
Room for growth
A
Actually in use
Operating system
(a)
B-Stack
Room for growth
B-Data
B-Program
A-Stack
Room for growth
A-Data
A-Program
Operating system
(b)

ניהול זיכרון באמצעות מפת סיביות



ניהול זיכרון באמצעות רשימות מקושרות



שיטות מיפוי

First fit	מתחילים מראש הרשימה	מהיר
Next fit	ממשיכים ממקום העצירה של החיפוש האחרון	פחות מהיר
Best Fit	חיפוש החור הקטן המתאים ביותר	גרוע
Worse fit	חיפוש החור הגדול ביותר	גם גרוע
Quick fit	חלוקה לרשימות בגדלים שונים	מהיר, אבל בעייתי באיחוד בלוקים פנויים

שיטות מיפוי

	מפת סיביות	רשימה	Free list	Quick fit רשימות בגדלים שונים
גודל	קטן מאוד	יותר גדול	לא תופס מקום	לא תופס
זמן חיפוש חור (הכי חשוב)	גדול	יותר מהיר	חיפוש ברשימה (סריקה)	מהיר מאוד
איחוד חורים	מהיר מאוד	מהיר	בינוני	איטי, יכול לגרום להעברת חורים מרשימה לרשימה

Swapping

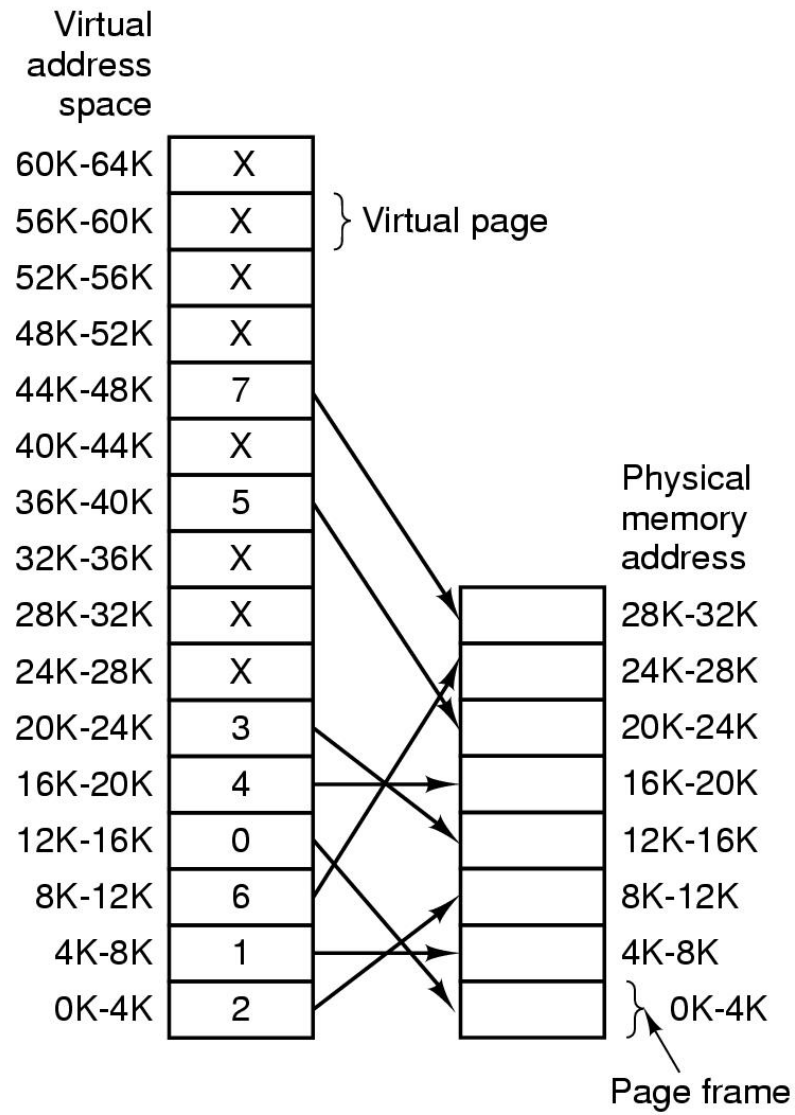
חסרונות:

- דורש זיכרון רציף
- כל התהליך שוכן או בזיכרון או בחוץ
- לא ניתן להריץ תהליך שדורש יותר מהחור הפנוי (או יותר מהזיכרון)
- Thrashing (סחרור)

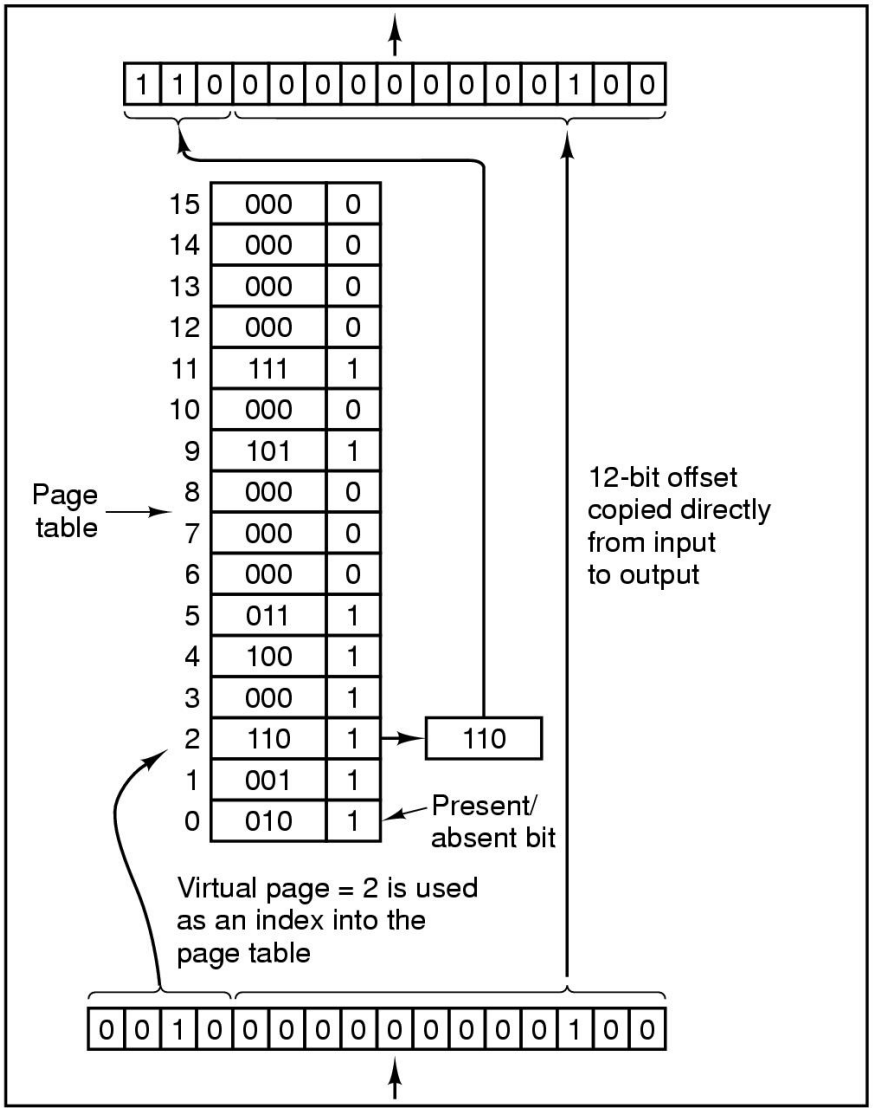
זיכרון וירטואלי

Virtual Memory

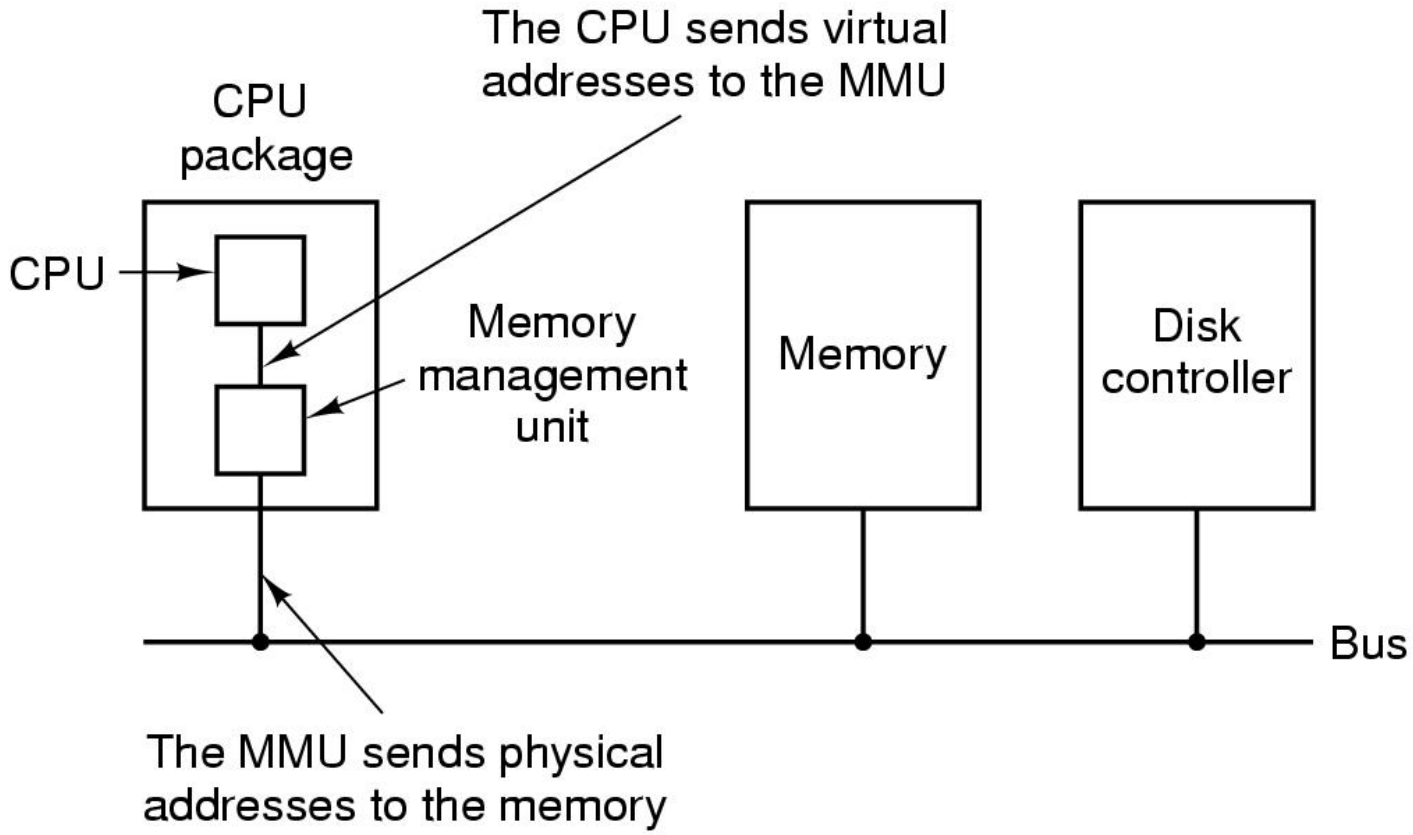
דפדוף


Virtual address space
60K-64K
56K-60K
52K-56K
48K-52K
44K-48K
40K-44K
36K-40K
32K-36K
28K-32K
24K-28K
20K-24K
16K-20K
12K-16K
8K-12K
4K-8K
0K-4K
X
X
X
X
7
X
5
X
X
3
4
0
6
1
2
Virtual page
Physical memory address
28K-32K
24K-28K
20K-24K
16K-20K
12K-16K
8K-12K
4K-8K
0K-4K
Page frame

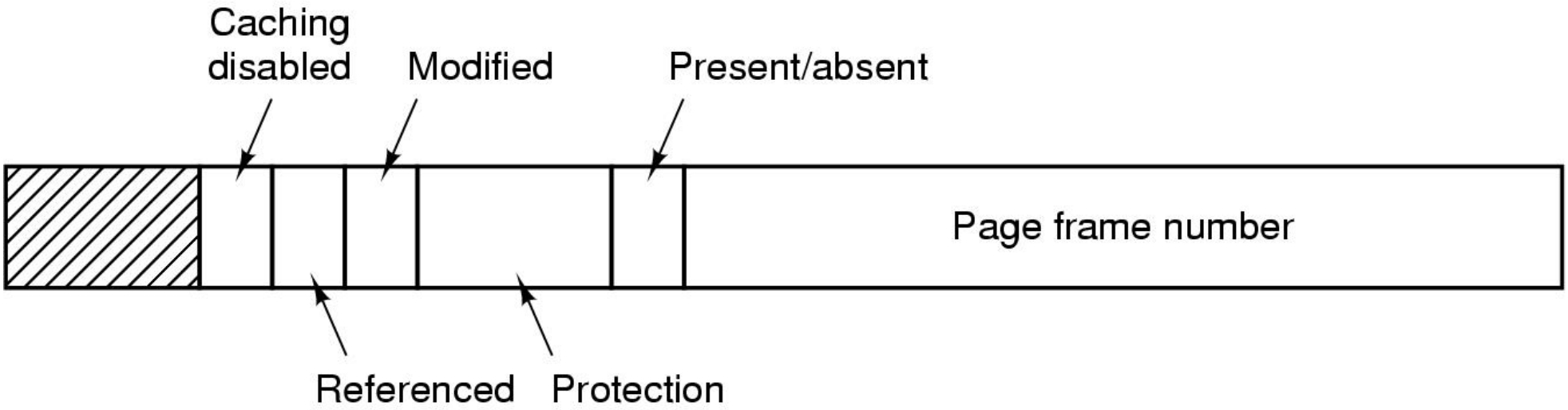
טבלת דפים


Outgoing physical address (24580)
1 1 0 0 0 0 0 0 0 0 0 0 1 0 0
15 000 0
14 000 0
13 000 0
12 000 0
11 111 1
10 000 0
9 101 1
8 000 0
7 000 0
6 000 0
5 011 1
4 100 1
3 000 1
2 110 1
1 001 1
0 010 1
Page table
12-bit offset copied directly from input to output
110
Present/ absent bit
Virtual page = 2 is used as an index into the page table
0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0
Incoming virtual address (8196)

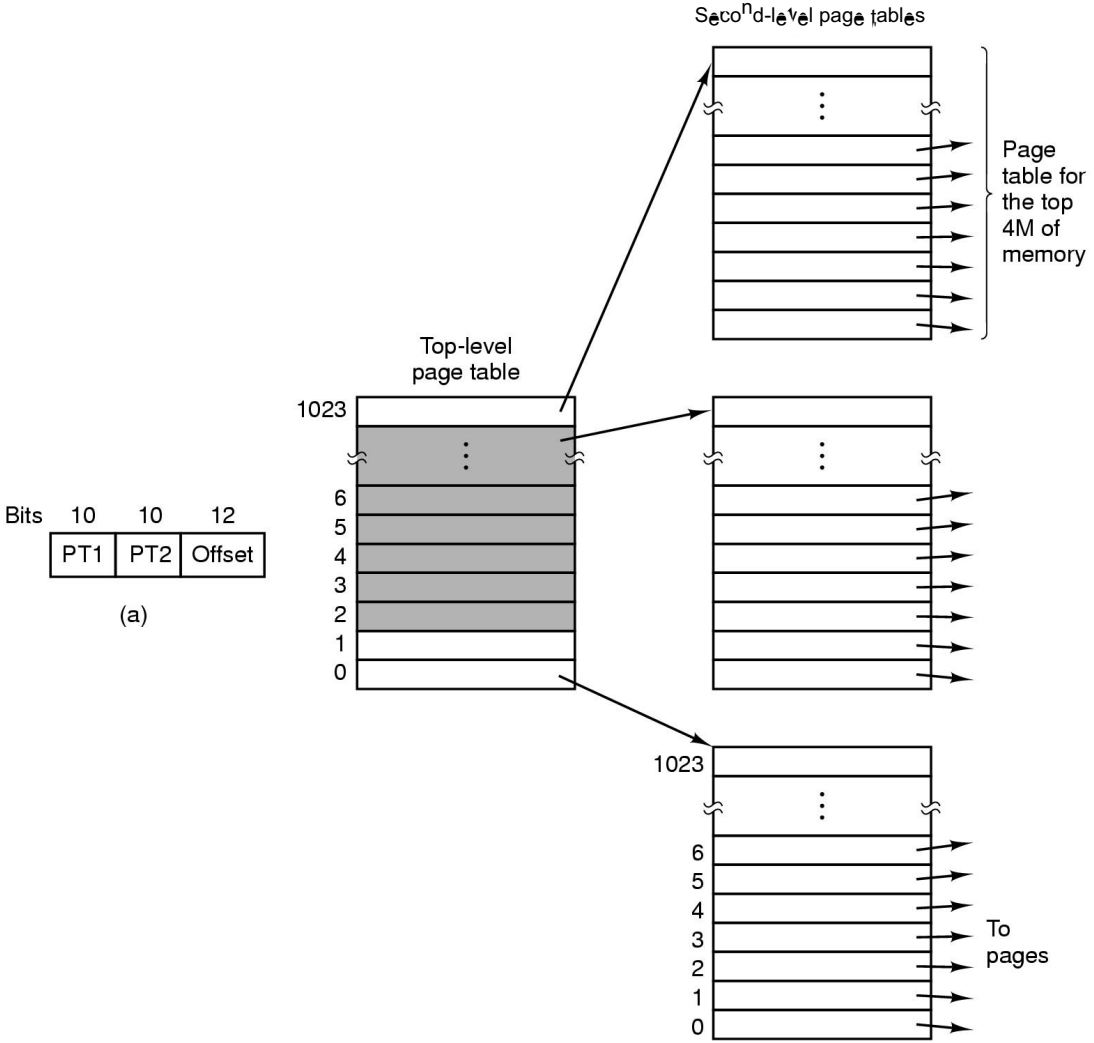
זיכרון וירטואלי



טבלת דפים


Caching disabled
Modified
Present/absent
Page frame number
Referenced
Protection

טבלת דפים


Second-level page tables
Page table for the top 4M of memory
Top-level page table
1023
6
5
4
3
2
1
0
Bits 10 10 12
PT1
PT2
Offset
(a)
1023
6
5
4
3
2
1
0
To pages

TLBs – Translation Lookaside Buffers

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

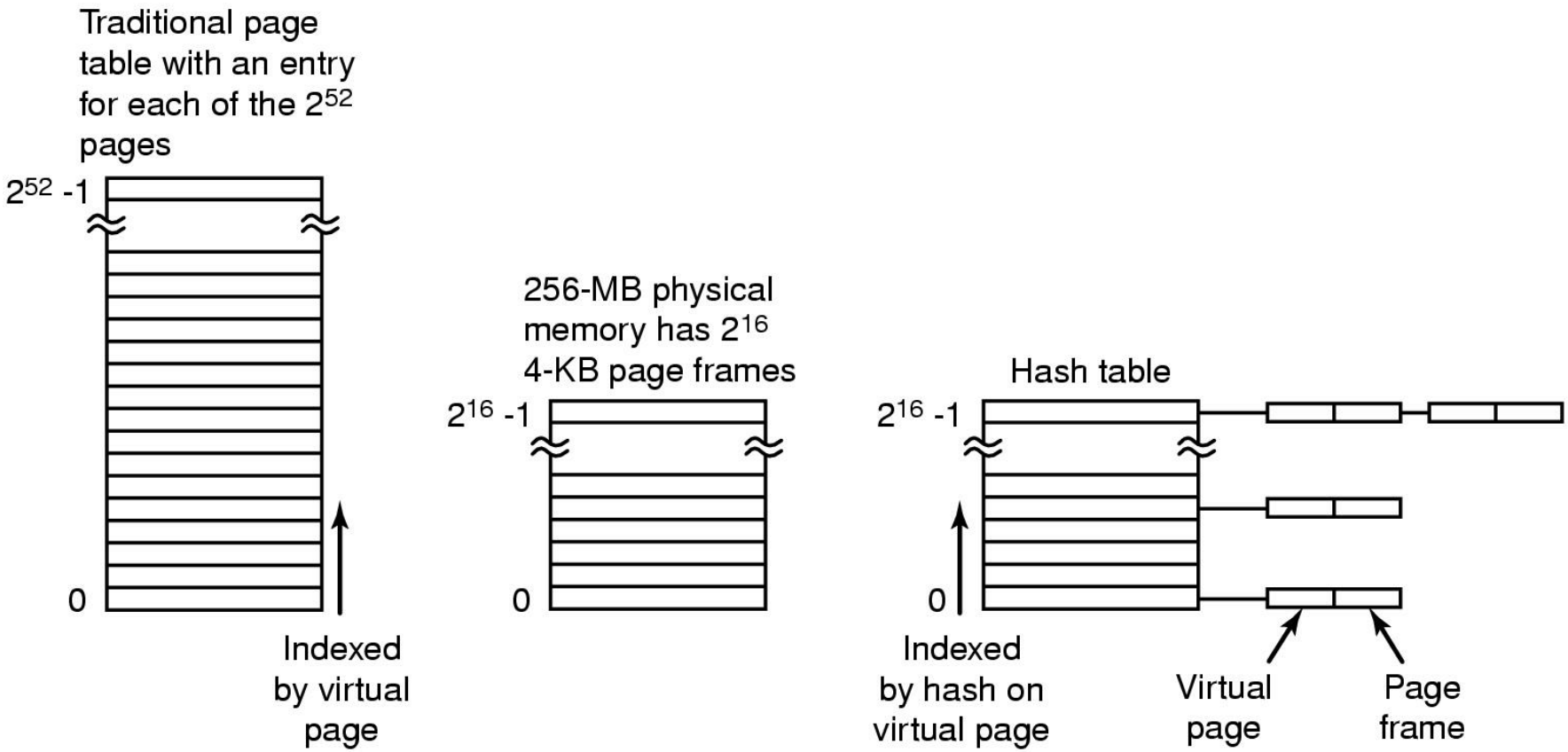
עקרון המקומיות

```
#define N 4096 // frame size
int main()
{
   char A[6][N];

   int i, j;
   for(i = 0; i < 6; i++)
      for(j = 0; j < N; j++)
         A[i][j] = 3;

   return 0;
}
```

טבלת דפים מהופכת



אלגוריתמי
ם להחלפת
דפים

Page Replacement
Algorithms

אלגוריתמים להחלפת דפים

- Optimal page replacement algorithm
- NRU, Not recently used page replacement
- FIFO, First-In, First-Out page replacement
- Second chance page replacement
- Clock page replacement
- LRU, Least recently used page replacement
- Working set page replacement
- WSClock page replacement

אלגוריתמים להחלפת דפים

- Page fault forces choice
  - which page must be removed
  - make room for incoming page
- Modified page must first be saved
  - unmodified just overwritten
- Better not to choose an often used page
  - will probably need to be brought back in soon

Optimal Page Replacement Algorithm

- Replace page needed at the farthest point in future
  - Optimal but unrealizable
- Estimate by …
  - logging page use on previous runs of process
  - although this is impractical

אלגוריתם - Not Recently Used

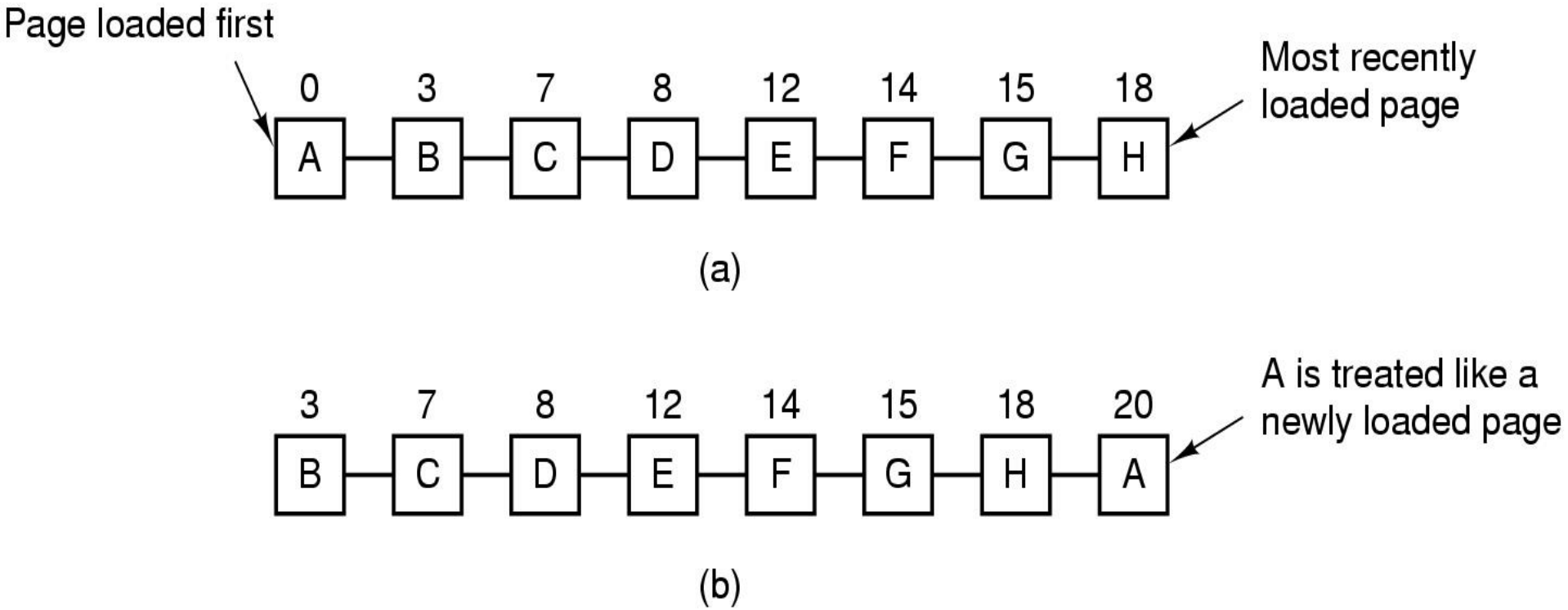
NRU

- Each page has Reference bit, Modified bit
  - bits are set when page is referenced, modified
- Pages are classified
  - not referenced, not modified
  - not referenced, modified
  - referenced, not modified
  - referenced, modified
- NRU removes page at random
  - from lowest numbered non empty class

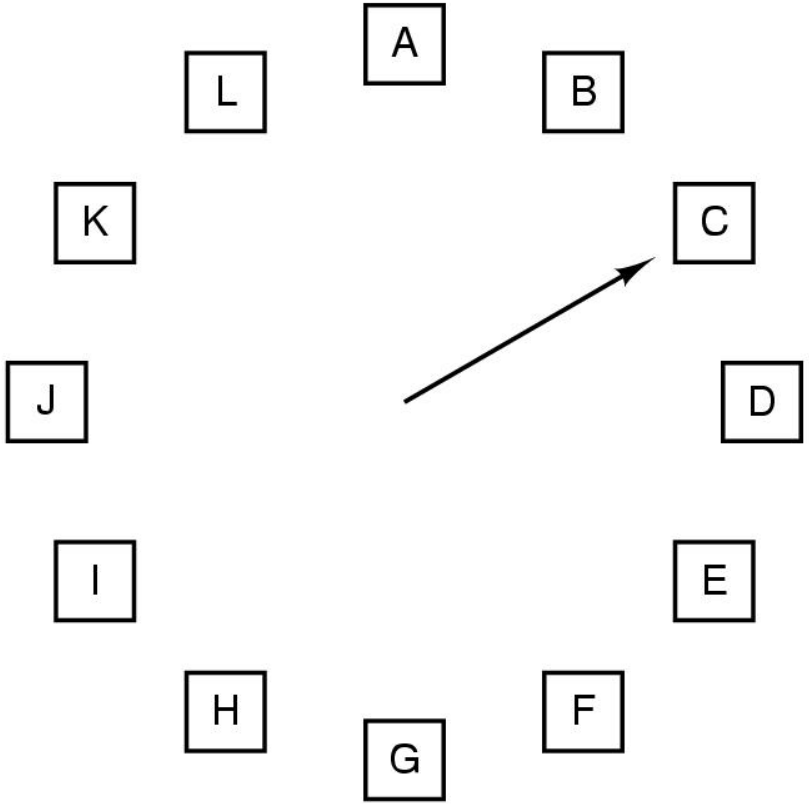
FIFO - תור

- Maintain a linked list of all pages
  - in order they came into memory
- Page at beginning of list replaced
- Disadvantage
  - page in memory the longest may be often used

הזדמנות שניה (second chance)



אלגוריתם השעון (clock page replacement)



When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

R = 0: Evict the page

R = 1: Clear R and advance hand

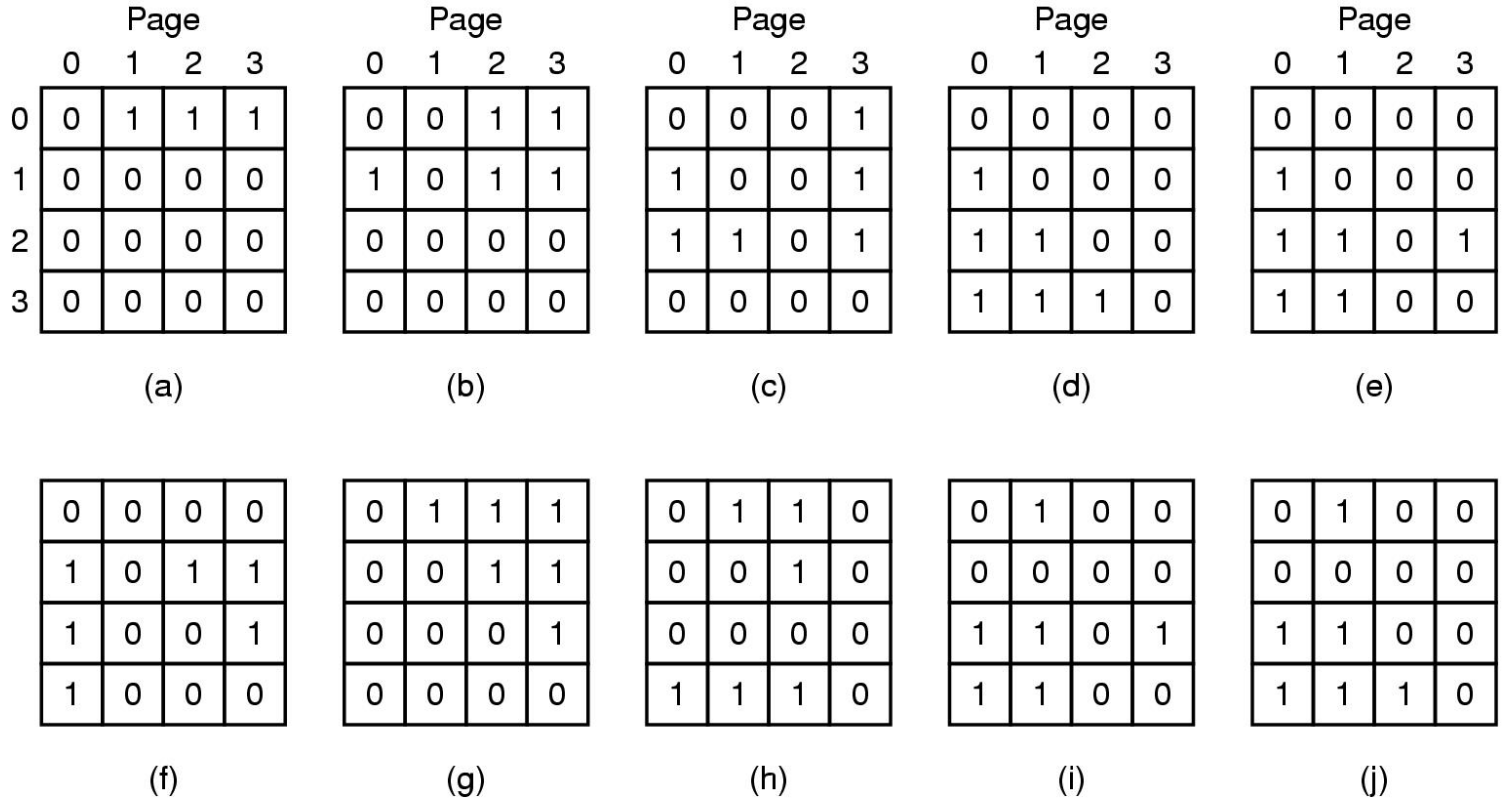
אלגוריתם – Least Recently Used

LRU

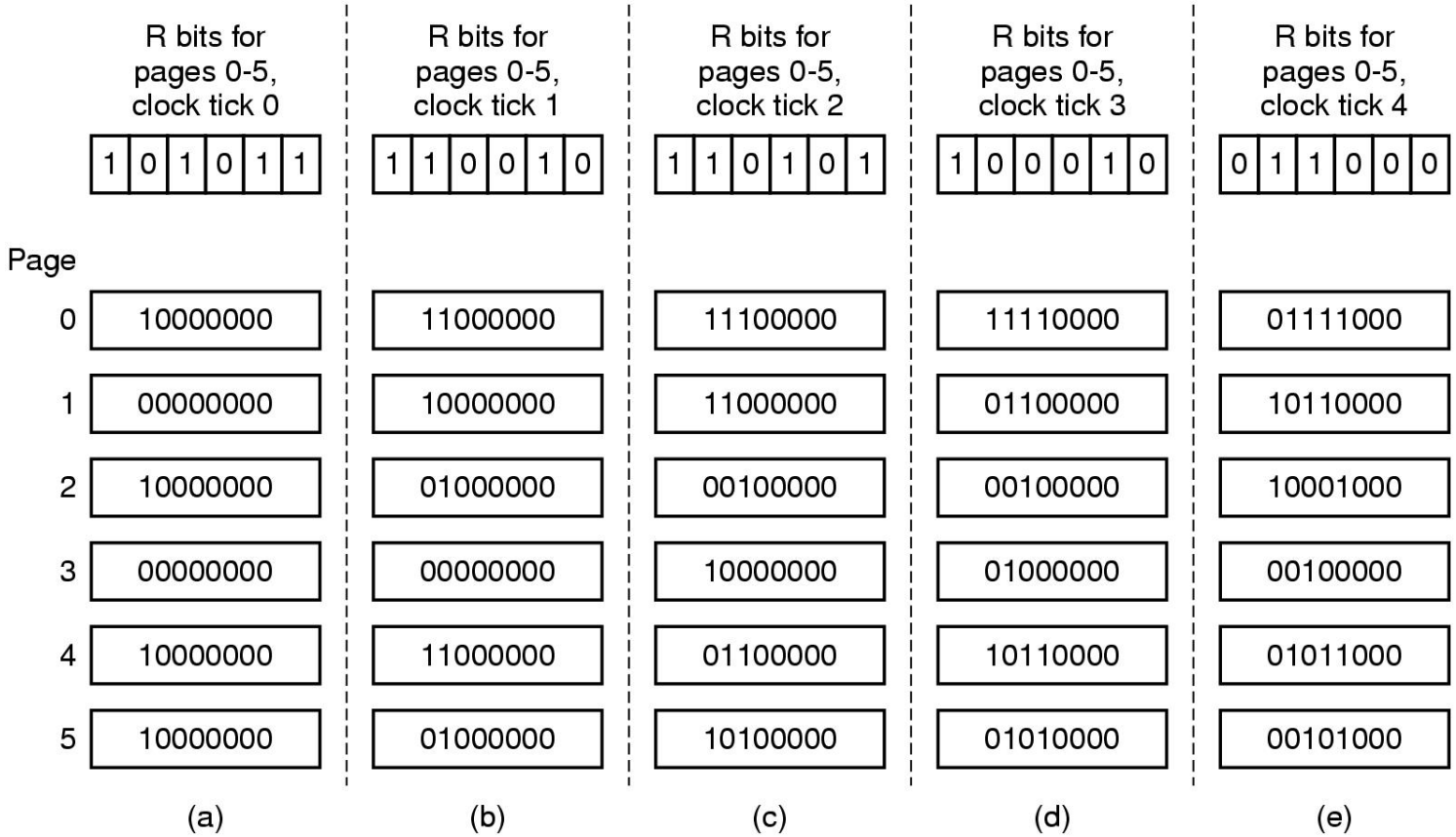
- Assume pages used recently will used again soon
  - throw out page that has been unused for longest time
- Must keep a linked list of pages
  - most recently used at front, least at rear
  - update this list every memory reference !!
- Alternatively keep counter in each page table entry
  - choose page with lowest value counter
  - periodically zero the counter

אלגוריתם – Least Recently Used

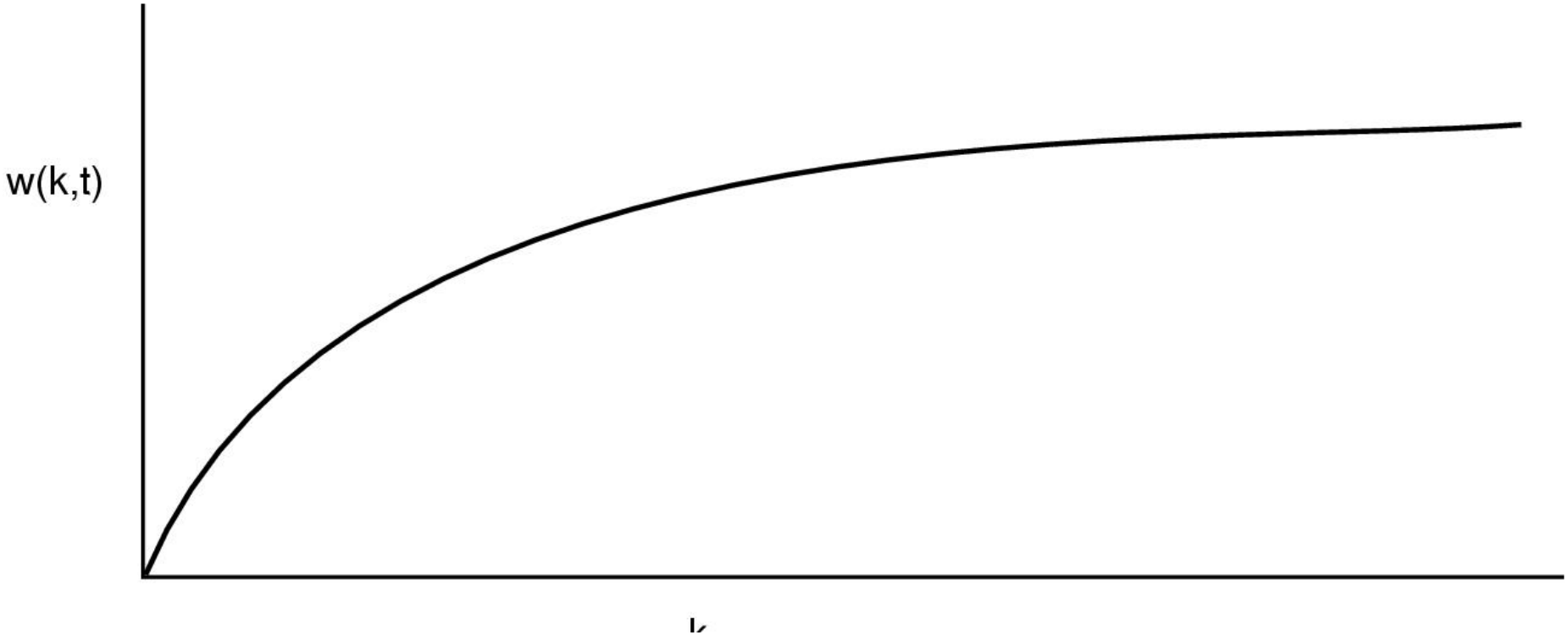
LRU



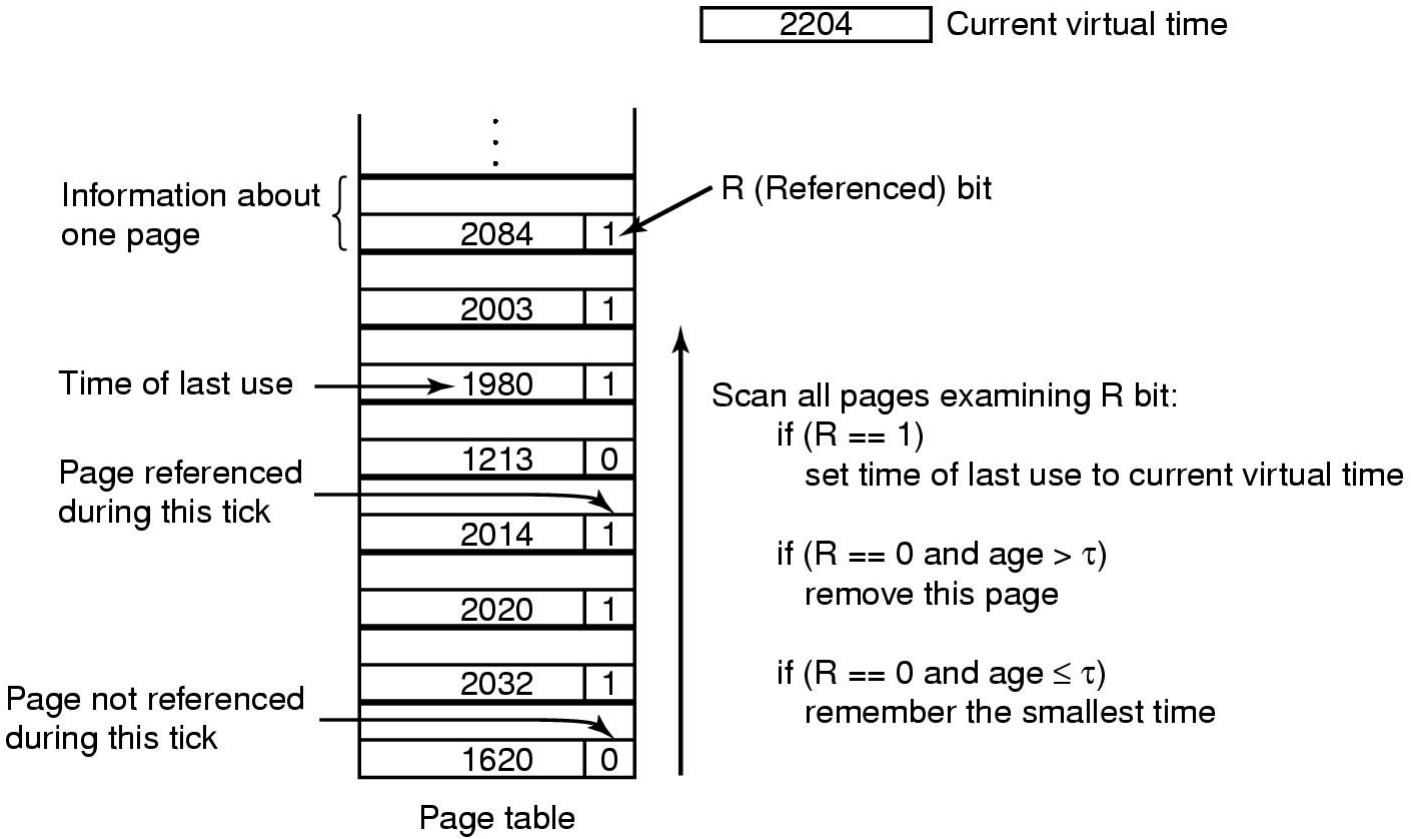
סימולציה של LRU בתוכנה



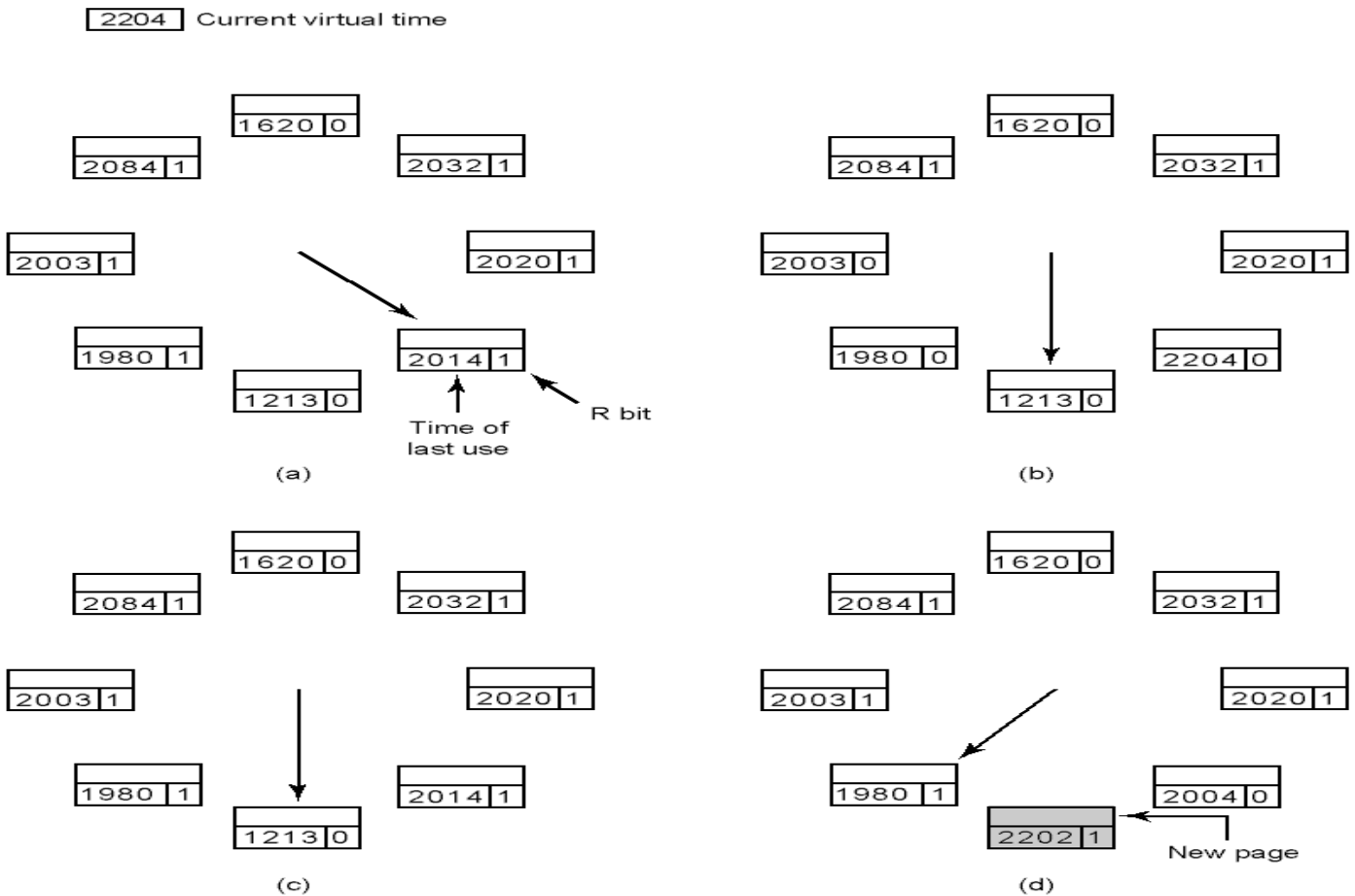
אלגוריתם המסתמך על קבוצת עבודה של תהליך



אלגוריתם המסתמך על קבוצת עבודה של תהליך



אלגוריתם WSClock



אלגוריתמים להחלפת דפים - סיכום

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Very crude
FIFO (First-In, First-Out)	Might throw out important pages
Second chance	Big improvement over FIFO
Clock	Realistic
LRU (Least Recently Used)	Excellent, but difficult to implement exactly
NFU (Not Frequently Used)	Fairly crude approximation to LRU
Aging	Efficient algorithm that approximates LRU well
Working set	Somewhat expensive to implement
WSClock	Good efficient algorithm

שיקולים בתכנון של מערכות דפדוף בזיכרון

Design Issues for Paging Systems

מדיניות החלפה מקומית לעומת גלובאלית

	Age
A0	10
A1	7
A2	5
A3	4
A4	6
A5	3
B0	9
B1	4
B2	6
B3	2
B4	5
B5	6
B6	12
C1	3
C2	5
C3	6

(a)

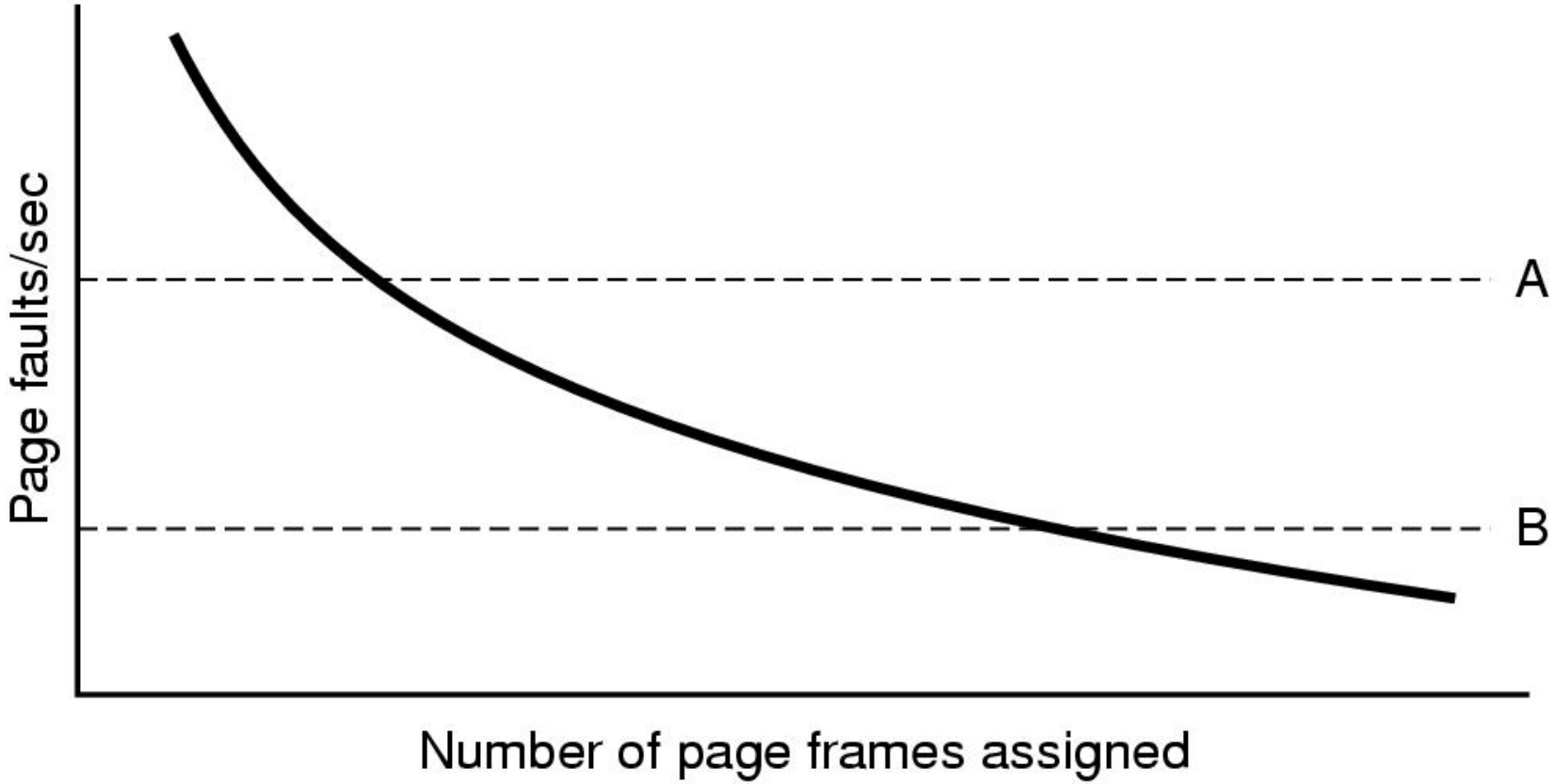
A0
A1
A2
A3
A4
A6
B0
B1
B2
B3
B4
B5
B6
C1
C2
C3

(b)

A0
A1
A2
A3
A4
A5
B0
B1
B2
A6
B4
B5
B6
C1
C2
C3

(c)

אלגוריתם PFF (Page Fault Frequency)



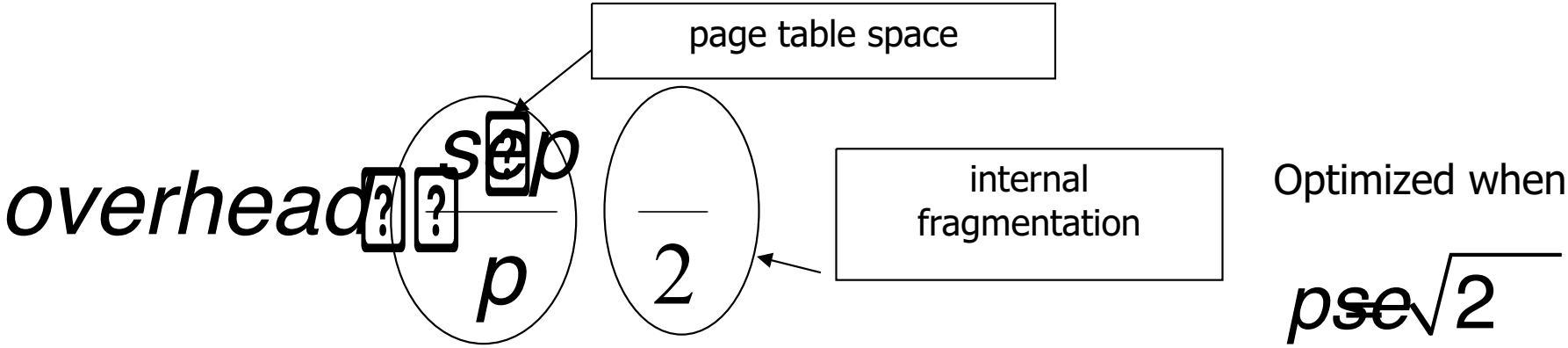
Load Control

- Despite good designs, system may still thrash
- When PFF algorithm indicates
  - some processes need more memory
  - but no processes need less
- Solution :
  Reduce number of processes competing for memory
  - swap one or more to disk, divide up pages they held
  - reconsider degree of multiprogramming

שיקולים לקביעת גודלו של הדף

- Overhead due to page table and internal fragmentation

- Where
  - s = average process size in bytes
  - p = page size in bytes
  - e = page entry



Optimized when

$p = \sqrt{2se}$

שיקולים לקביעת גודלו של הדף

- דפים קטנים
- דפים גדולים

$$overhead = \frac{seek}{p} \cdot \frac{p}{2}$$

Optimized when

$$p = \sqrt{2 \cdot seek}$$

דפים לשימוש משותף

Cleaning Policy

- Need for a background process, paging daemon
  - periodically inspects state of memory
- When too few frames are free
  - selects pages to evict using a replacement algorithm
- It can use same circular list (clock)
  - as regular page replacement algorithm but with diff ptr

Implementation Issues

1. Process creation
   - determine program size
   - create page table
2. Process execution
   - MMU reset for new process
   - TLB flushed
3. Page fault time
   - determine virtual address causing fault
   - swap target page out, needed page in
4. Process termination time
   - release page table, pages

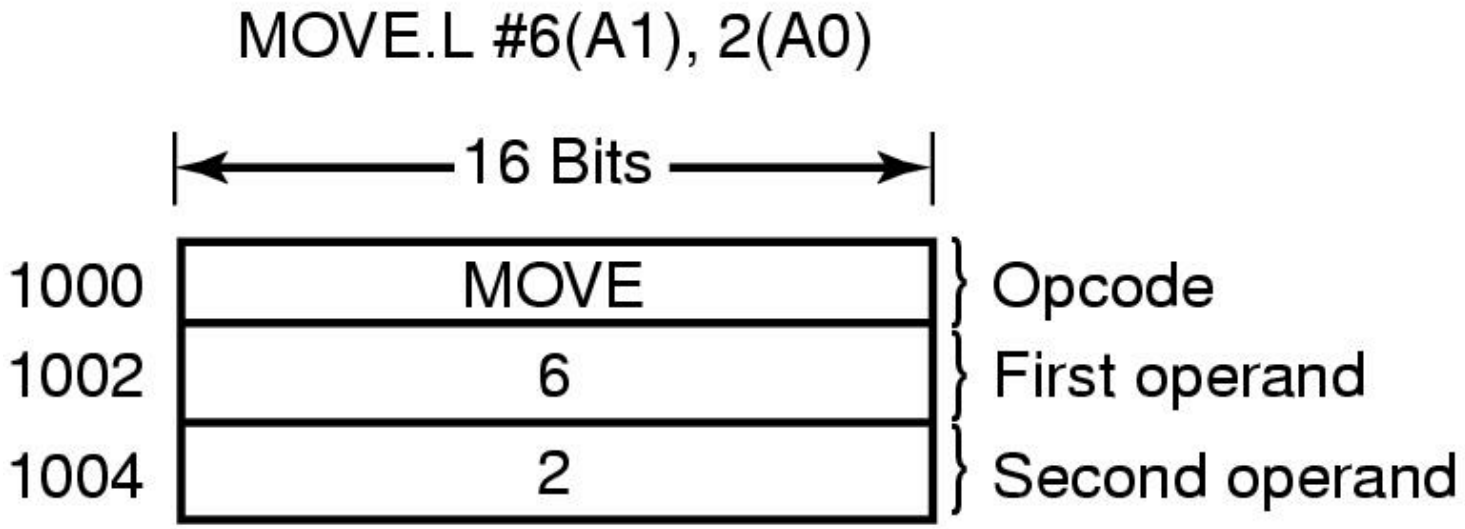
סיכום דרך הטיפול בשגיאת דף

1. Hardware traps to kernel
2. General registers saved
3. OS determines which virtual page needed
4. OS checks validity of address, seeks page frame
5. If selected frame is dirty, write it to disk
6. OS brings schedules new page in from disk
7. Page tables updated

סיכום דרך הטיפול בשגיאת דף

2. OS finds a space in memory for the page
3. OS determines which virtual page needed
4. OS checks validity of address, seeks page frame
5. If selected frame is dirty, write it to disk
6. OS brings schedules new page in from disk
7. Page tables updated
8. Faulting instruction backed up to when it began
9. Faulting process scheduled
10. Registers restored
11. Program continues

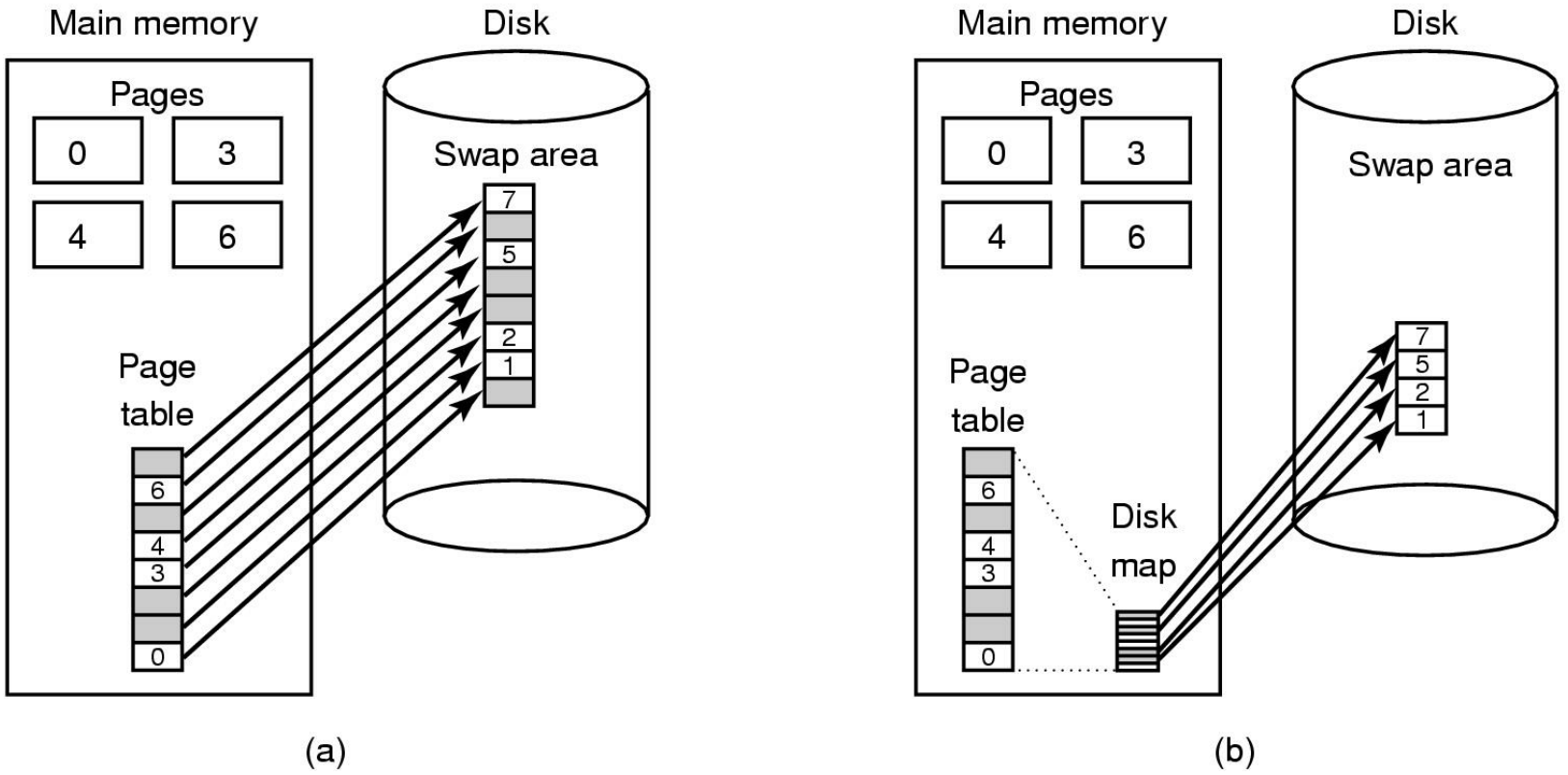
שחזור הוראות מכונה לאחר פסיקת דף



Locking Pages in Memory

- Virtual memory and I/O occasionally interact
- Proc issues call for read from device into buffer
  - while waiting for I/O, another processes starts up
  - has a page fault
  - buffer for the first proc may be chosen to be paged out
- Need to specify some pages locked
  - exempted from being target pages

Backing Store

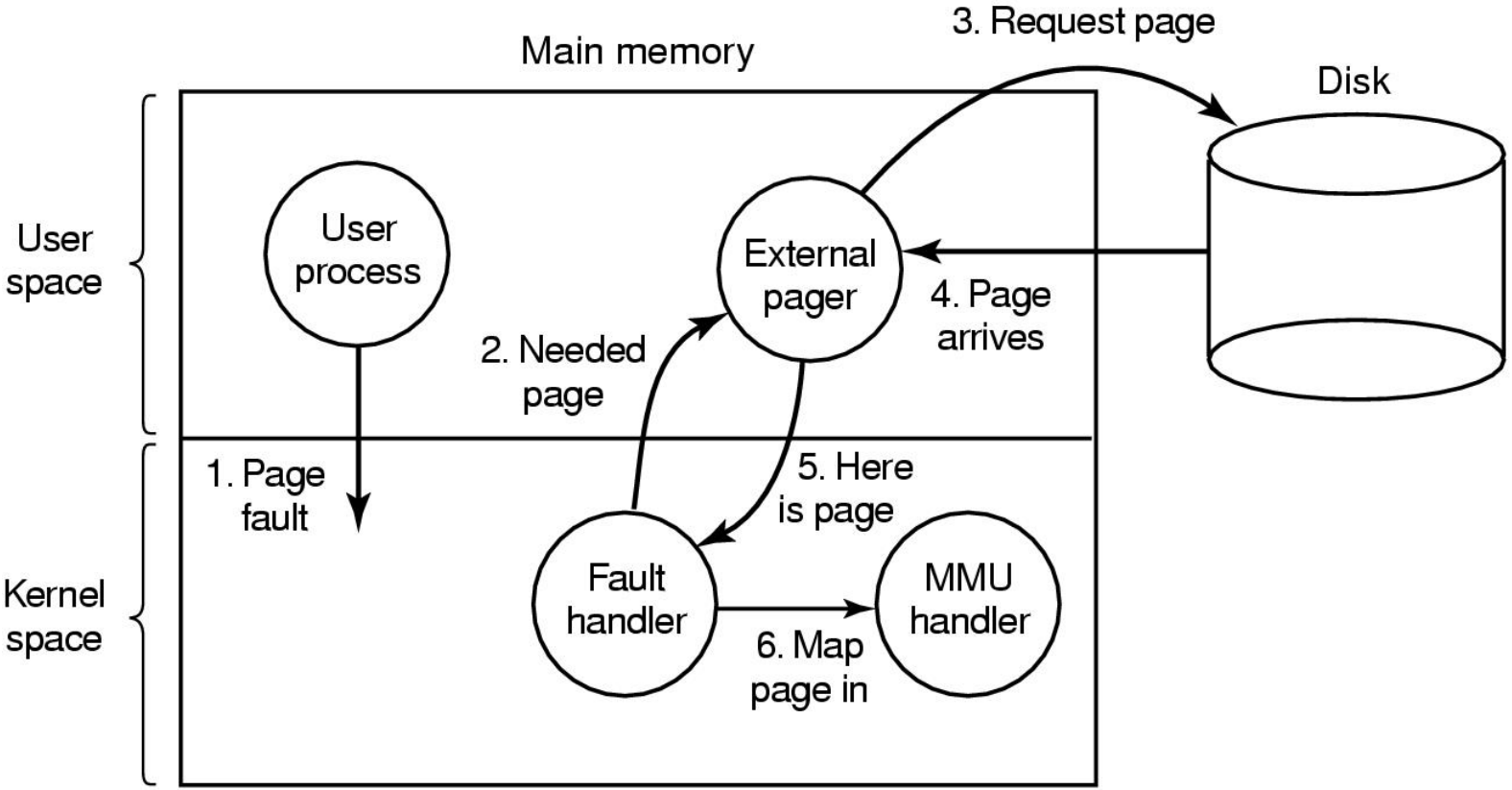


Separation of Policy and Mechanism

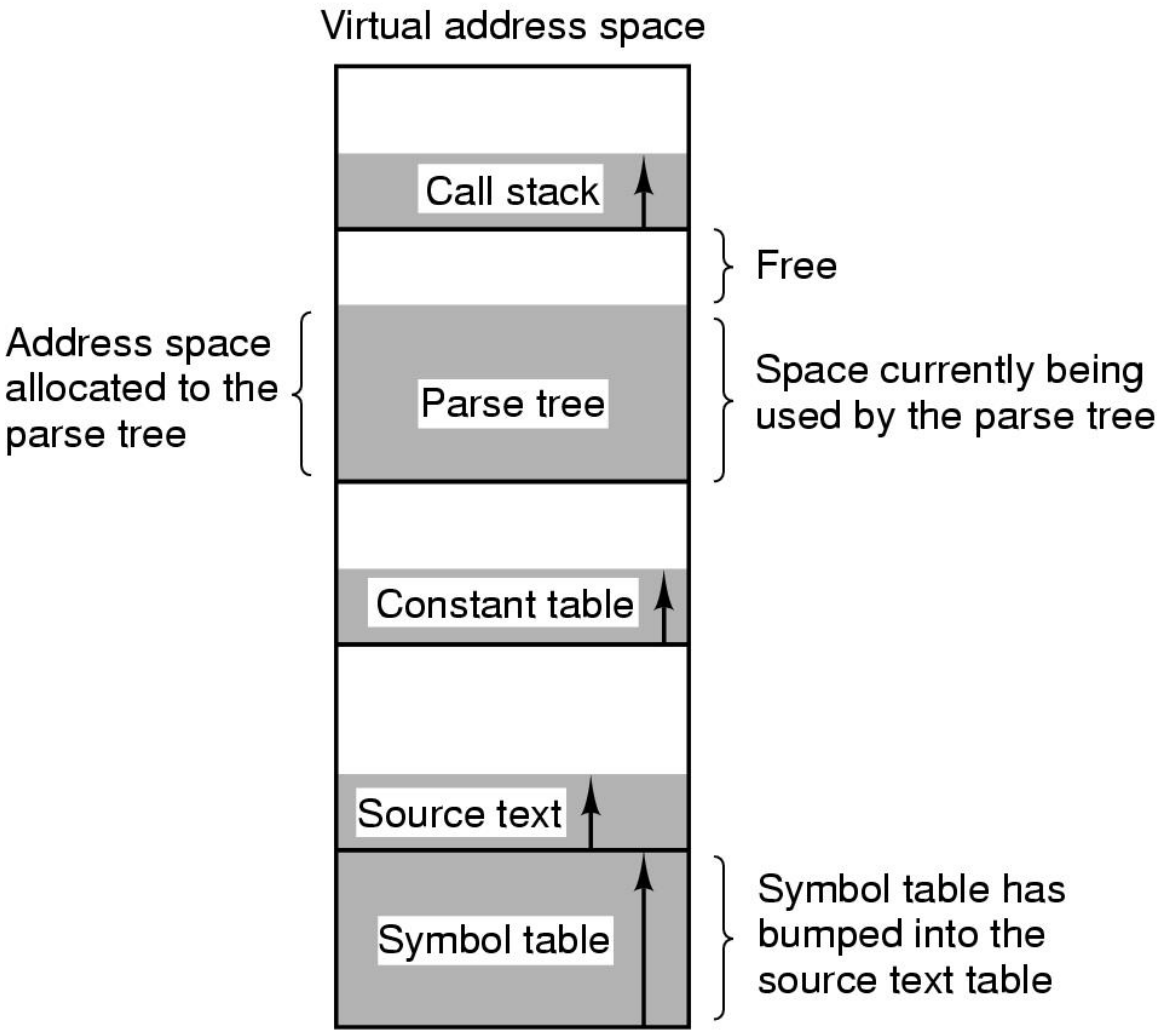
Memory management system is divided into three parts:

- A low-level MMU handler
- A page fault handler that is part of the kernel
- An external pager running in user space

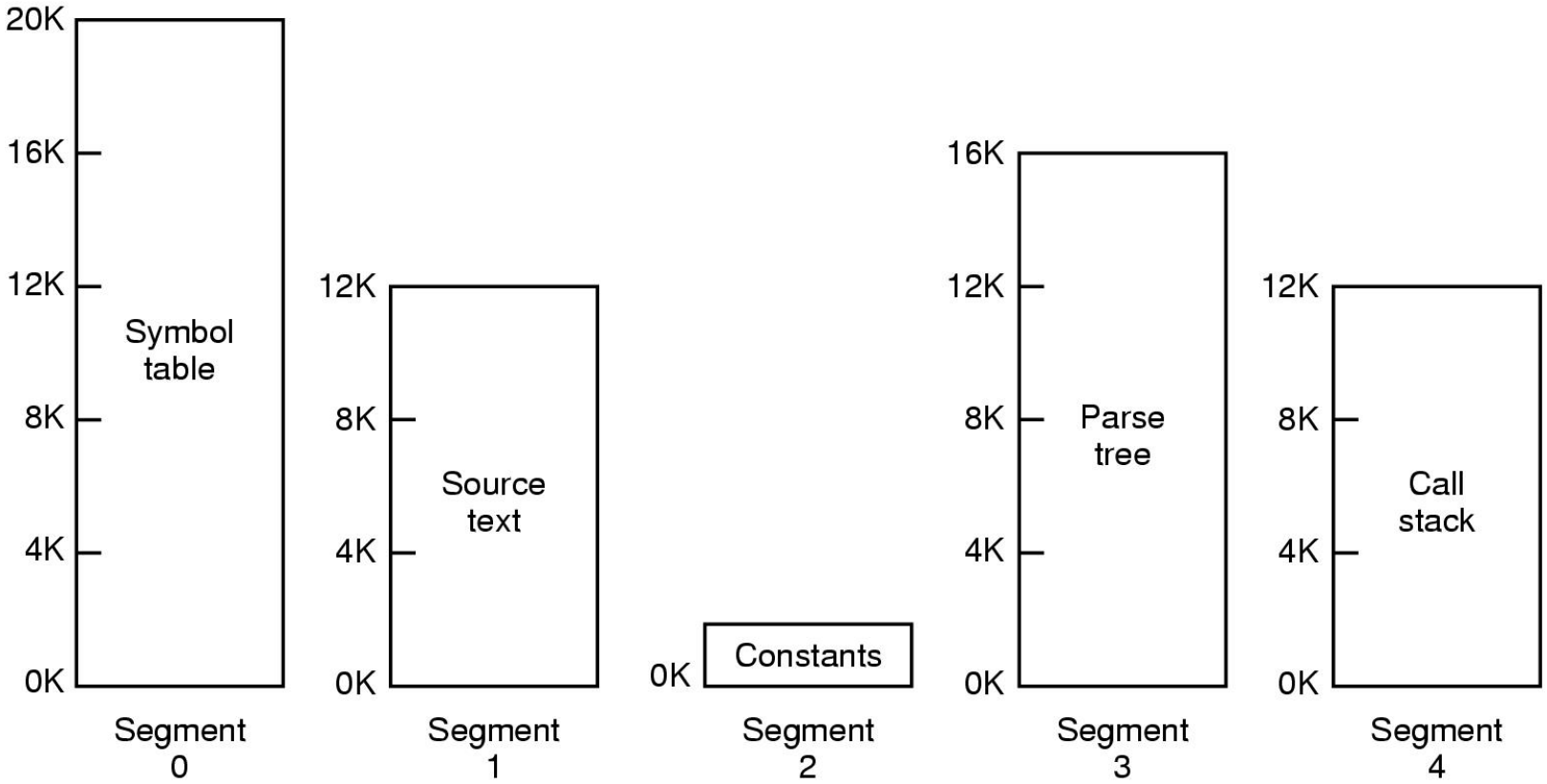
Separation of Policy and Mechanism



Segmentation



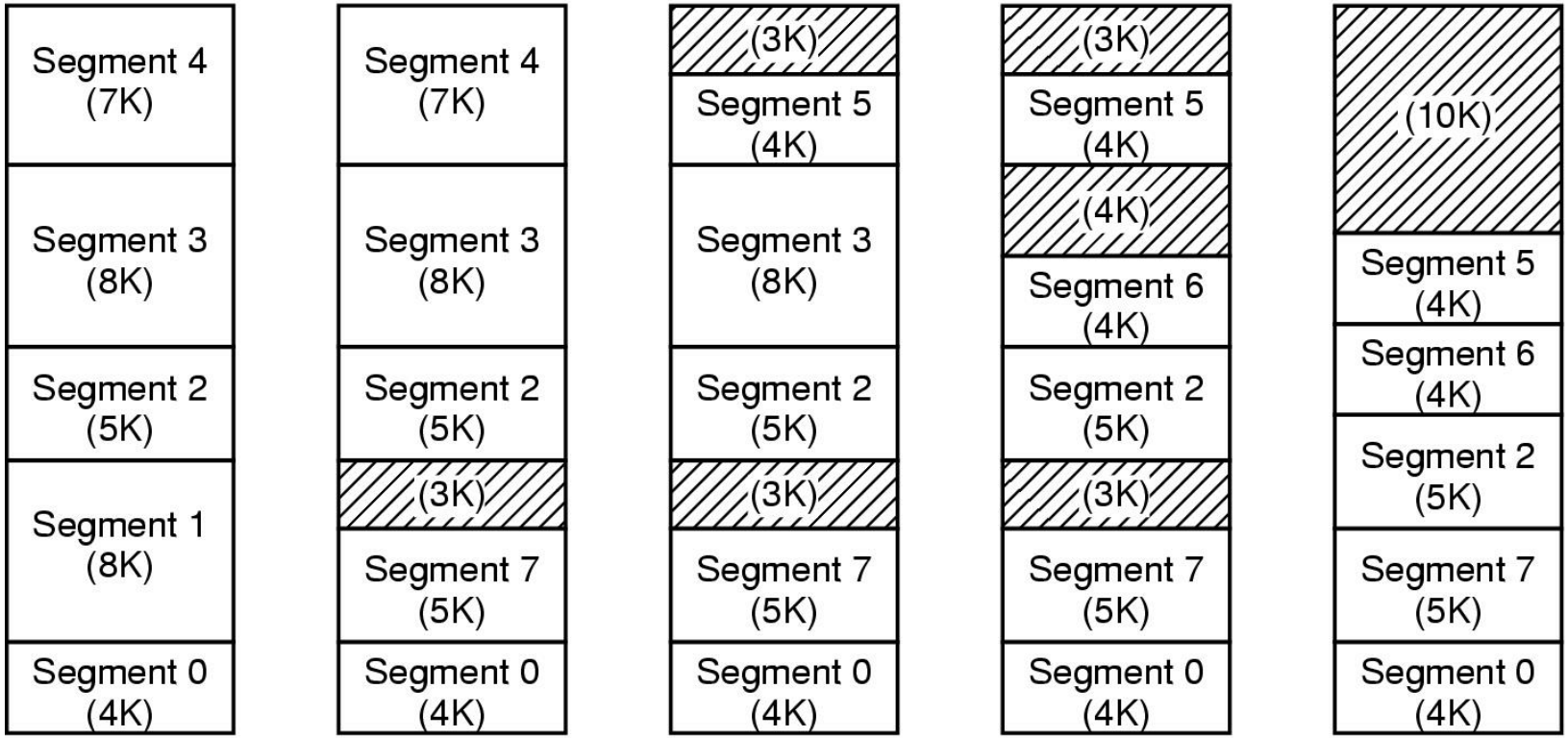
Segmentation



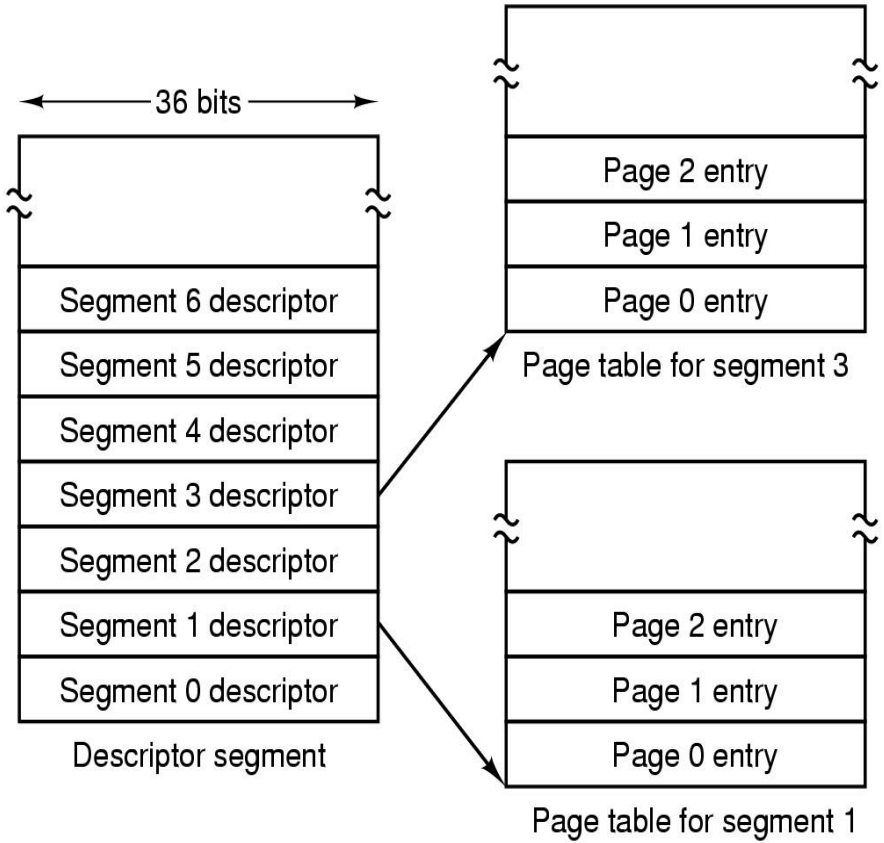
Segmentation

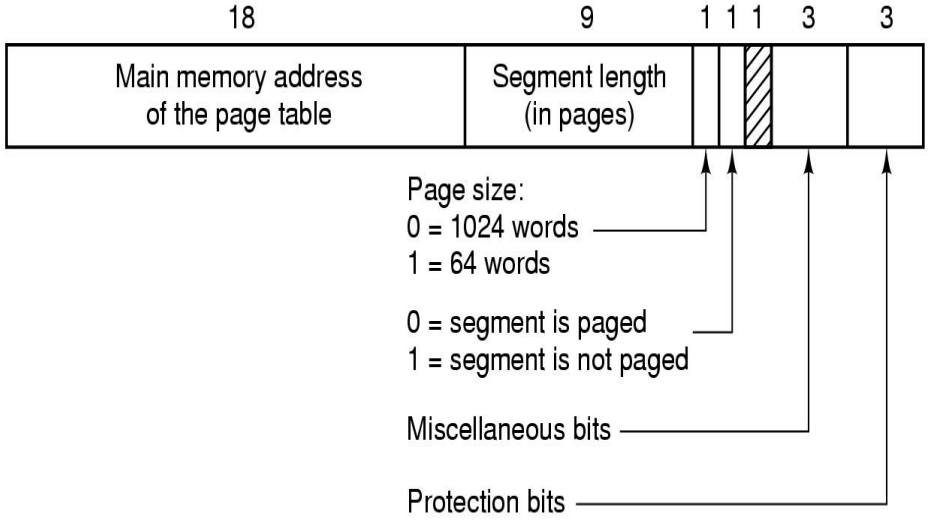
Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

Implementation of Pure Segmentation

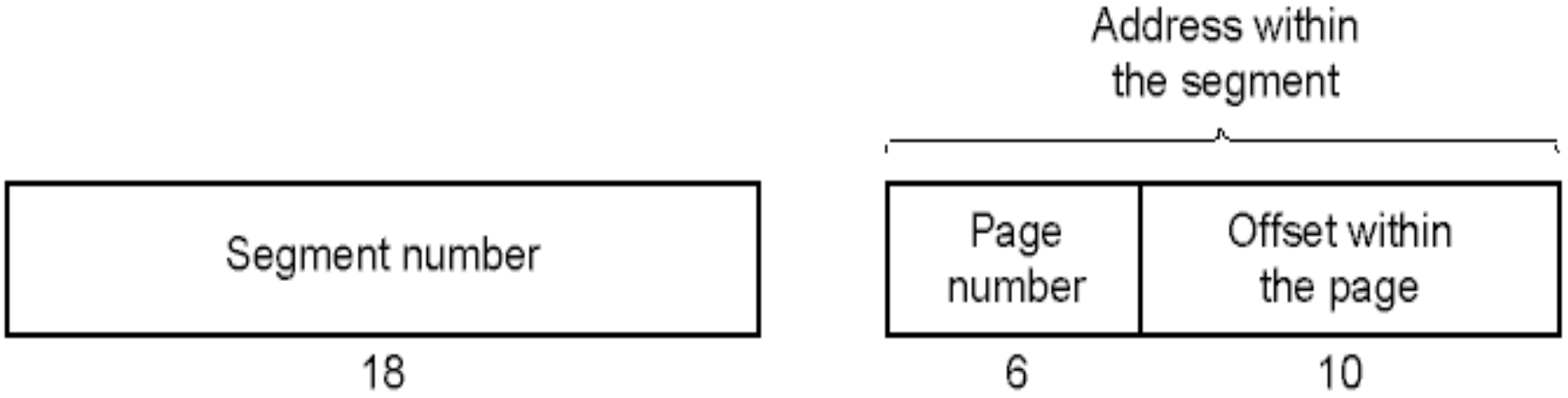


Segmentation with Paging: MULTICS (1)

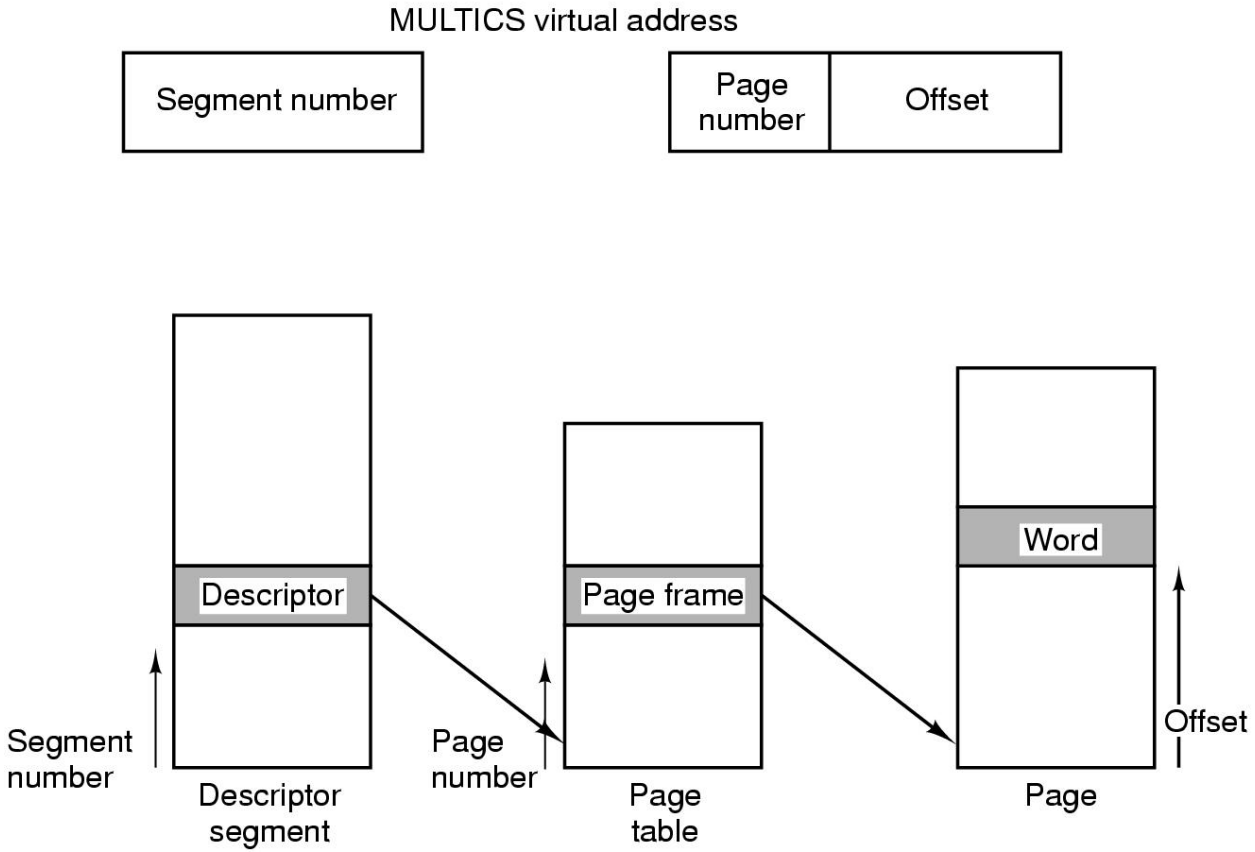




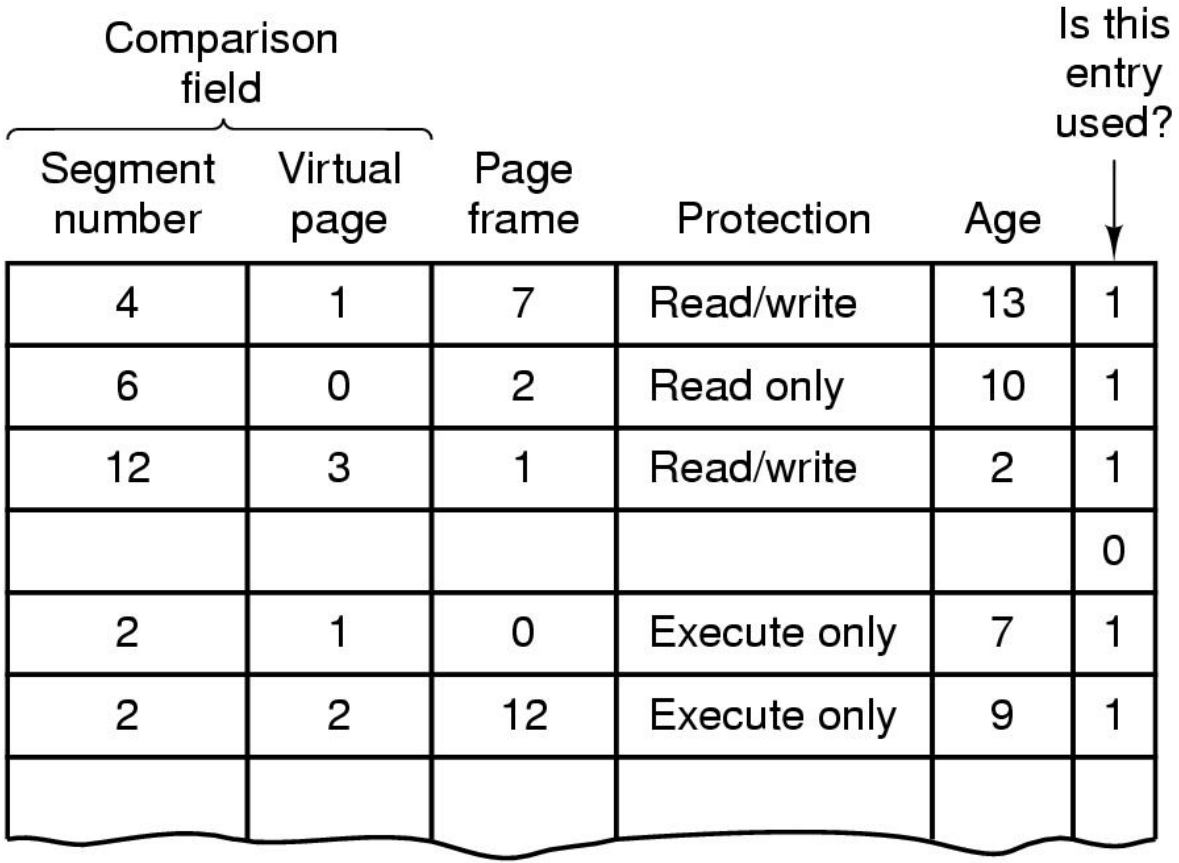
Segmentation with Paging: MULTICS (2)



Segmentation with Paging: MULTICS (3)



Segmentation with Paging: MULTICS (4)



Comparison field: Segment number	Comparison field: Virtual page	Page frame	Protection	Age	Is this entry used?
4	1	7	Read/write	13	1
6	0	2	Read only	10	1
12	3	1	Read/write	2	1
					0
2	1	0	Execute only	7	1
2	2	12	Execute only	9	1

תרגול

שאלה 1

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון

- גודל הדף הוא 2 בתים
- כתובת וירטואלית בת 16 סיביות כאשר 10 סיביות מיועדת להיסט בתוך הדף (offset)

מהו גודל מילת הזיכרון

א. 1 byte

ב. 2 bytes

ג. 4 bytes

ד. אף תשובה לא נכונה

שאלה 2

נתון כי:

- במערכת עם ניהול זיכרון באמצעות הדפדוף (paging)
- הינה בת 16 (virtual address) כתובת מדומה
- גודל הדף במערכת הוא 512 bytes
- גודל מילת זיכרון הוא 1 byte

מהי כמות הדפים במרחב הזיכרון המדומה

א. 128

ב. 256

ג. 512

שאלה 3

נתון:

- גודל דף 2 word
- word = 2 bytes
- טבלת הדפים הרגילה היא בדיוק בגודל דף
- גודל כניסה בטבלה 4 bytes

כמה סיביות יש בכתובת הוירטואלית?

שאלה 4

נתון כי במערכת מסוימת המשתמשת בשיטת חלוקת הזיכרון לקטעים בשיתוף עם דפדוף (segmentation with paging) גודל הכתובת הוירטואלית הוא 34 סיביות. הכתובת מחולקת ל 18 סיביות לזיהוי הקטע (segment) ו 16 סיביות לכתובת הפנימית בתוך הקטע עצמו. 16 הסיביות הללו מתחלקות ל 6 סיביות של מספר הדף ו 10 סיביות להיסט (offset):

segment number	page number	offset
18bits	6bits	10bits
most significant bits		least significant bits

להלן חלק של טבלת הקטעים (segment table) וחלקים של טבלאות הדפים (page tables) המתאימות:

Segment table

Segment table entry	Page table number
0	67
1	105

Page table number 67

Page table entry	Frame number
8	16
9	64

Page table number 105

Page table entry	Frame number
7	32
8	128

שאלה 5

נתון:

- 32 סיביות בכתובת הוירטואלית
- 12 סיביות להיסט
- 1 הצבעות אדף מכיל

א. כמה דפים צריכים בשביל להחזיק את הטבלה (טבלה רגילה)?

ב. מה גודל הטבלה במילים (word)?

שאלה 6

נתון כי:

- גודל הדף הוא 4 KB
- כניסה של טבלת דפים בכל שיכבה היא בת 4 bytes
- אורך הכתובת המדומה היא בת 64 סיביות
- אורך מילת זיכרון הוא 2 bytes

מהי כמות השכבות המינימאלית שתידרש על מנת ששהטבלאות ייכנסו לדף אחד?

א. 3
ב. 4
ג. 5
ד. 6

עושה מערכת עם ניהול זיכרון באמצעות דפדוף (paging) שימוש בטבלת דפים שכבתית (multilevel page table).

סיכום

סיכום



ניהול זיכרון

- בסיסי
- מחיצות נעות
- החלפה (swapping)
- זיכרון וירטואלי



אלגוריתמים להחלפת דפים

- אופטימלי



שיקולים בתכנון של מערכות דפדוף בזיכרון

- מדיניות החלפה
- אלגוריתם PFF
- גודל דף
- אופטימלי



תודה